DEVOPS

LAB 3

Working with Git History, Stashing, and Reverting Commits

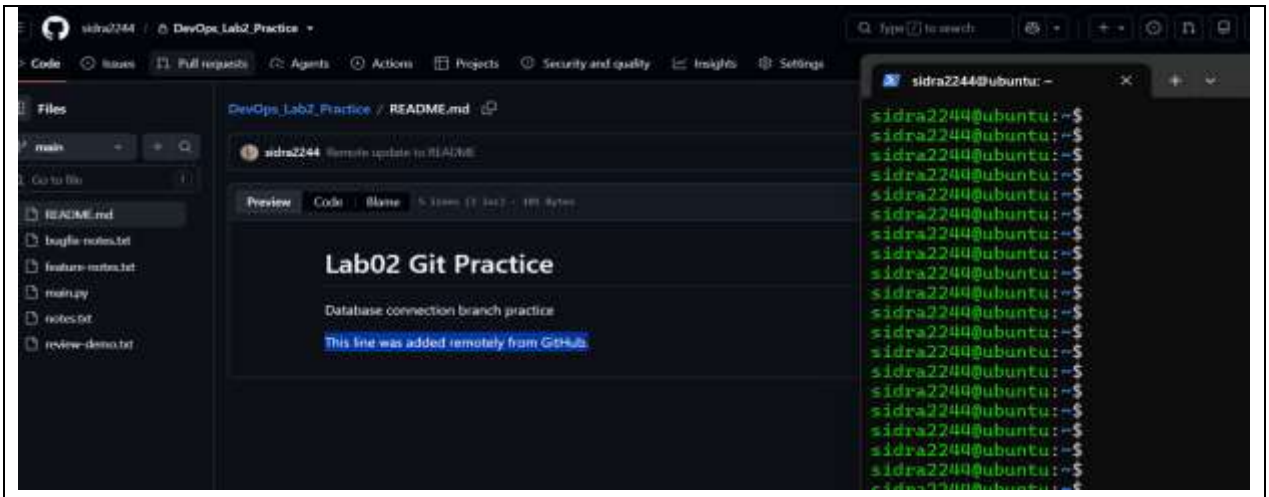
Student Name	Sidra-Tul-Muntaha
Registration Number	__23-BSCS-084___
Course Title / Code	DevOps
GitHub Username	sidra2244
Submission Repository	DevOps
GitHub Repository Link	https://github.com/sidra2244/DevOps_Lab2_Practice

Task 1 – Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase)

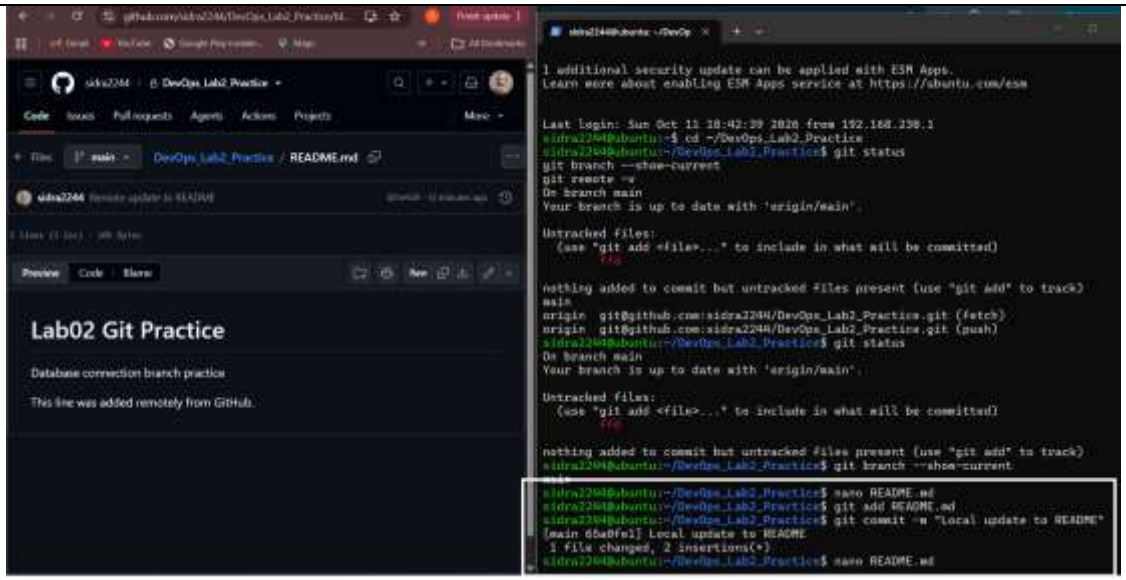
This task demonstrates how Git handles commits made separately on GitHub and in the local repository. A normal merge combines histories, while rebase replays local commits on top of the updated remote history.

Evidence and Explanation

Screenshot: remote_edit.png

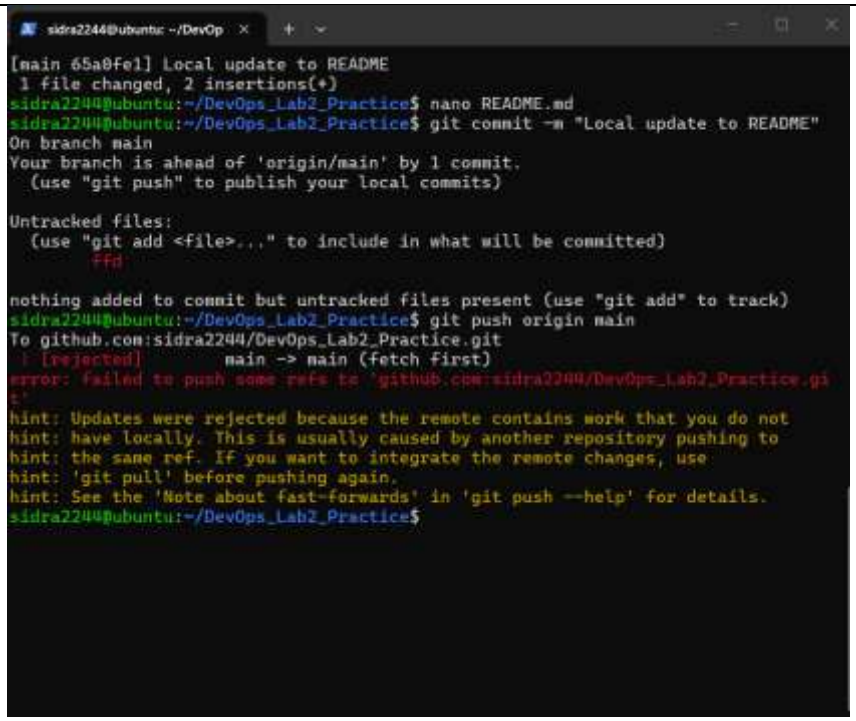


Screenshot: local_commit.png



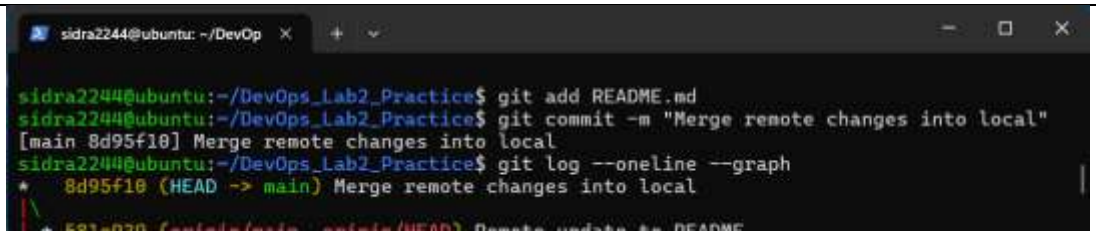
A separate local README.md change was staged and committed.

Screenshot: push_error.png



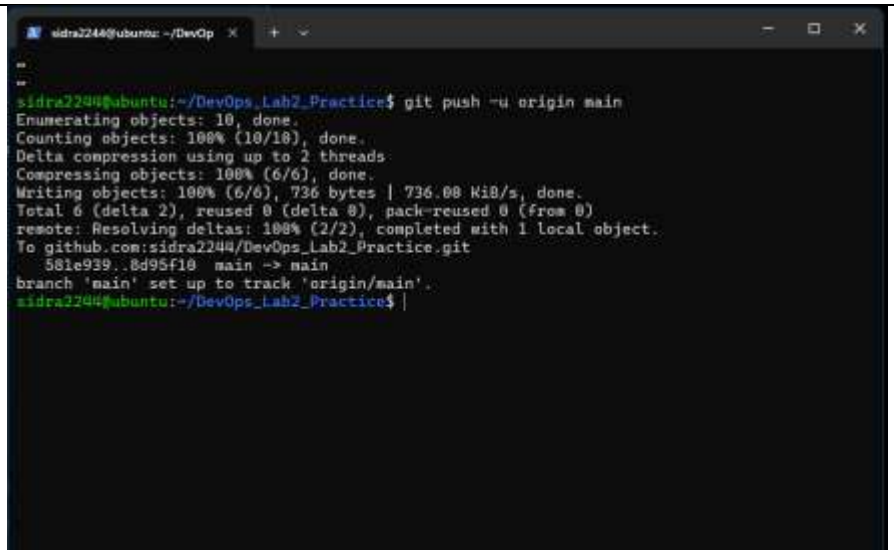
The push was rejected because the remote repository contained commits not present locally.

Screenshot: merge_commit.png



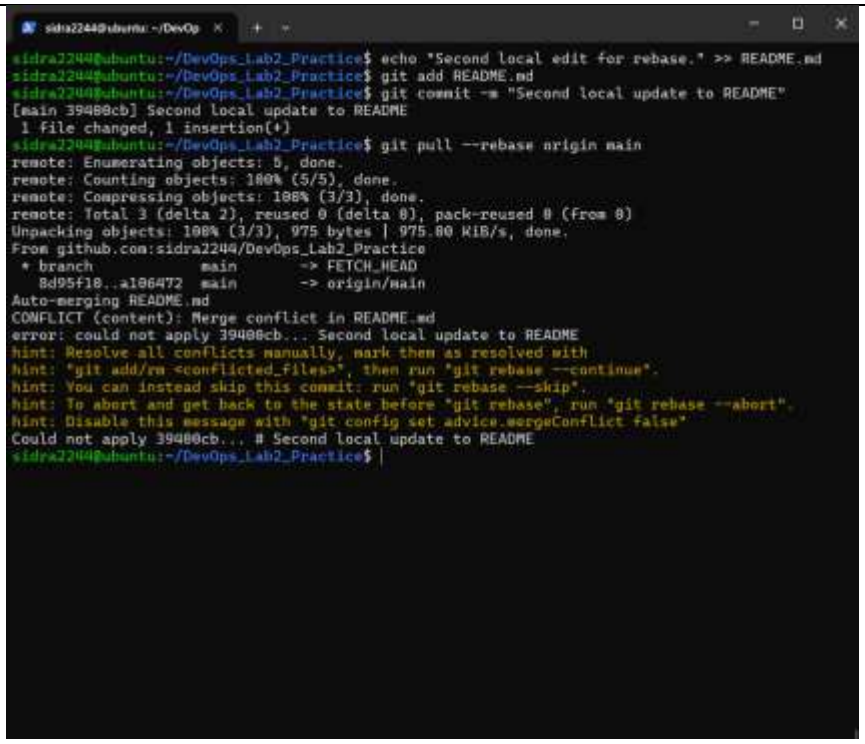
The remote changes were pulled using --no-rebase and merged with the local work.

Screenshot: push_after_merge.png



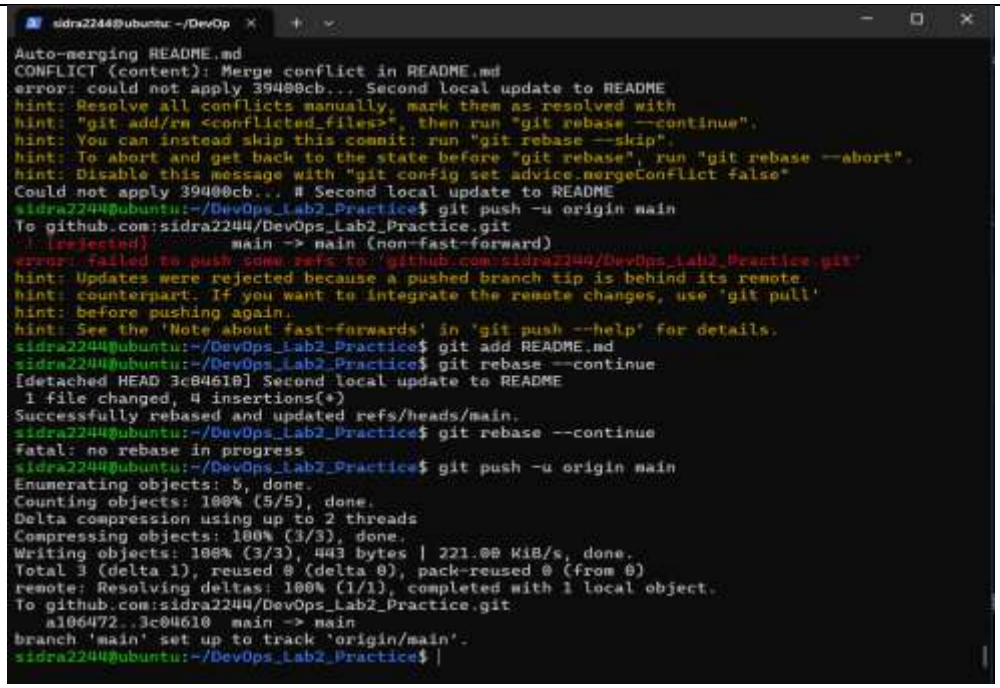
The merged history was pushed successfully to GitHub.

Screenshot: rebase_pull.png



The rebase workflow replayed the local commit on top of the latest remote commit.

Screenshot: push_after_rebase.png



The rebased history was pushed to the remote repository.

Task Summary

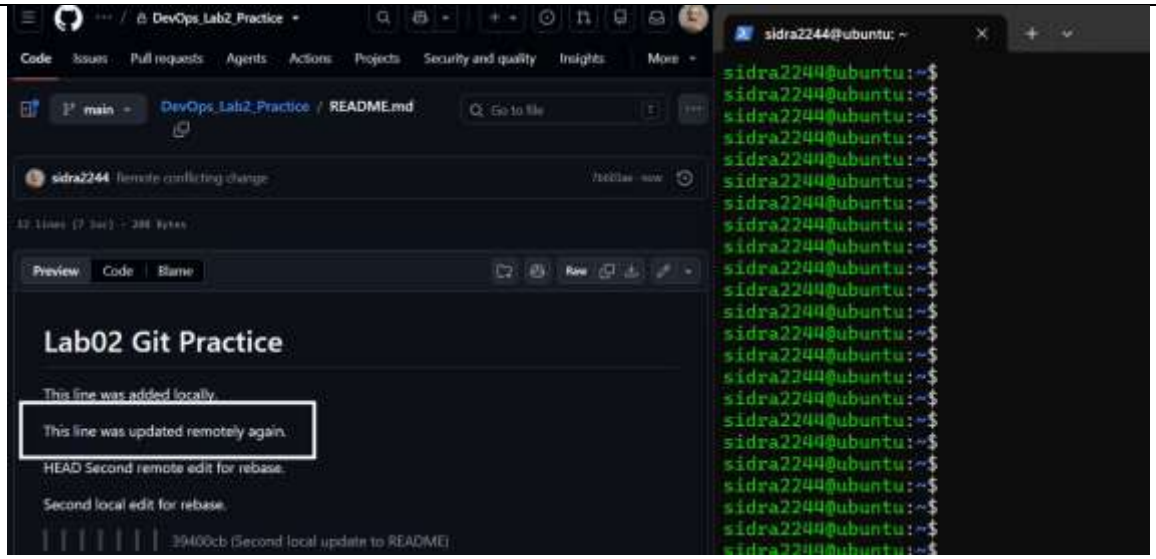
git pull --no-rebase uses merge, whereas git pull --rebase keeps the commit history linear.

Task 2 – Creating and Resolving Merge Conflicts Manually

This task creates a conflict by changing the same line differently on GitHub and locally. Git pauses the rebase so the file can be edited and the conflict resolved manually.

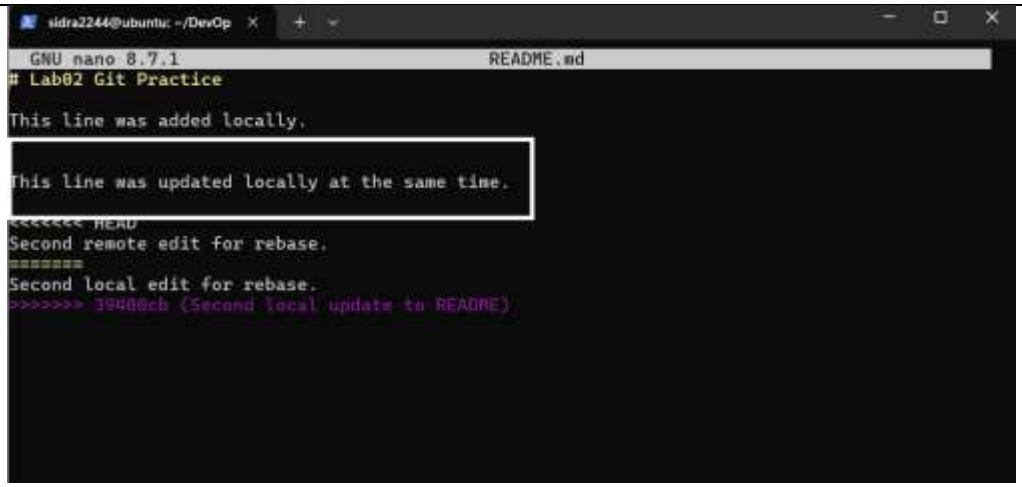
Evidence and Explanation

Screenshot: remote_conflict_edit.png



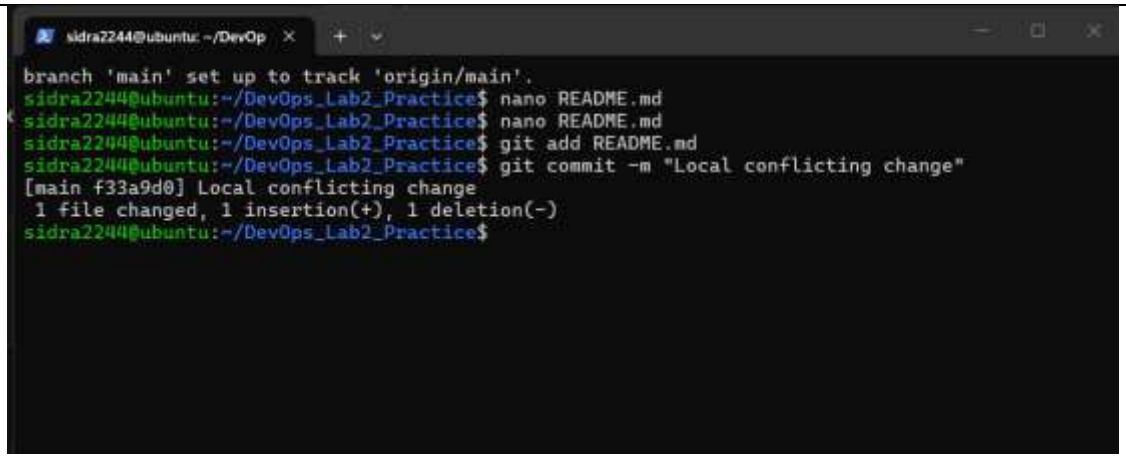
The target README.md line was changed on GitHub.

Screenshot: local_conflict_edit.png



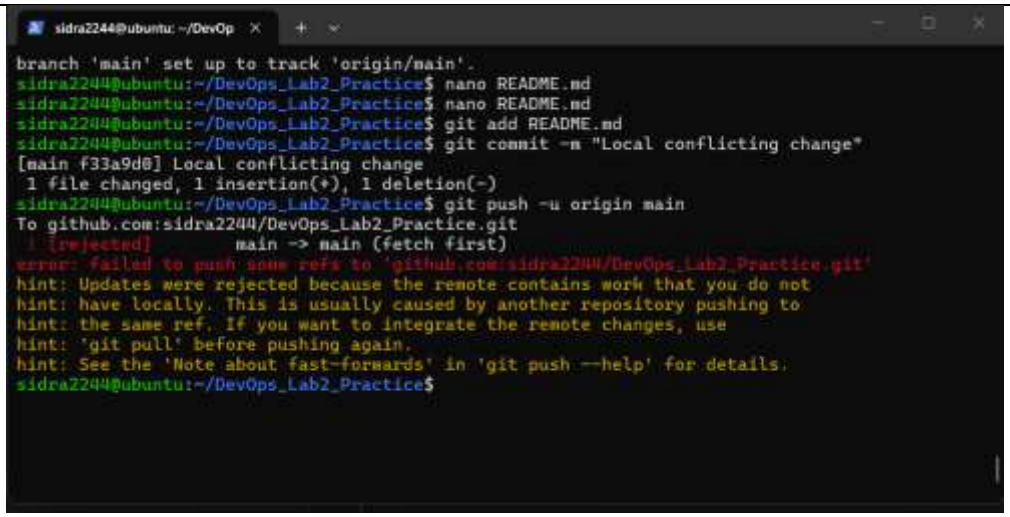
The same line was changed differently in the local file.

Screenshot: local_conflict_commit.png



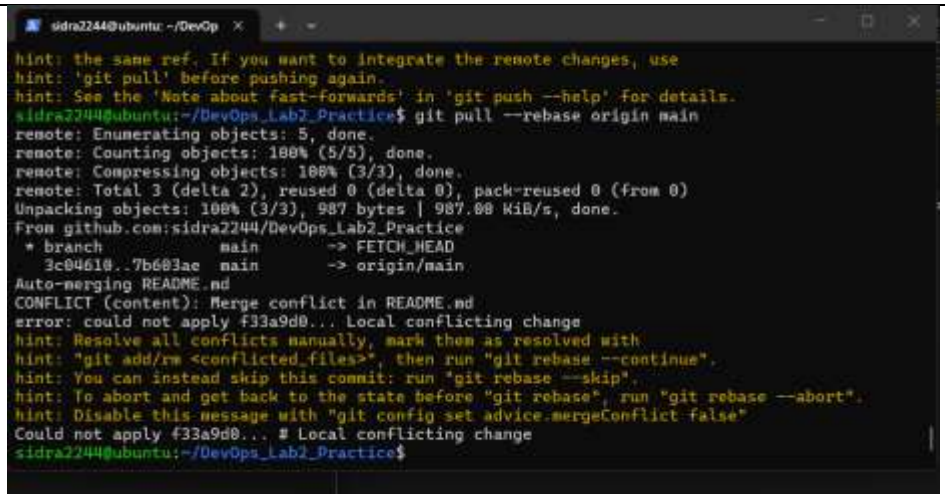
The local conflicting change was committed.

Screenshot: conflict_push_error.png



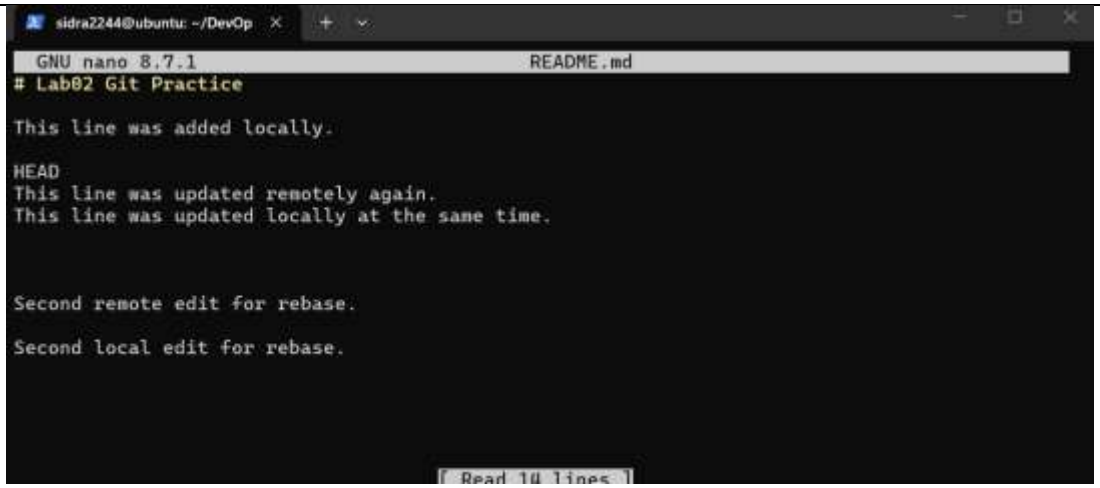
The push was rejected because the remote branch had a conflicting update.

Screenshot: conflict_message.png



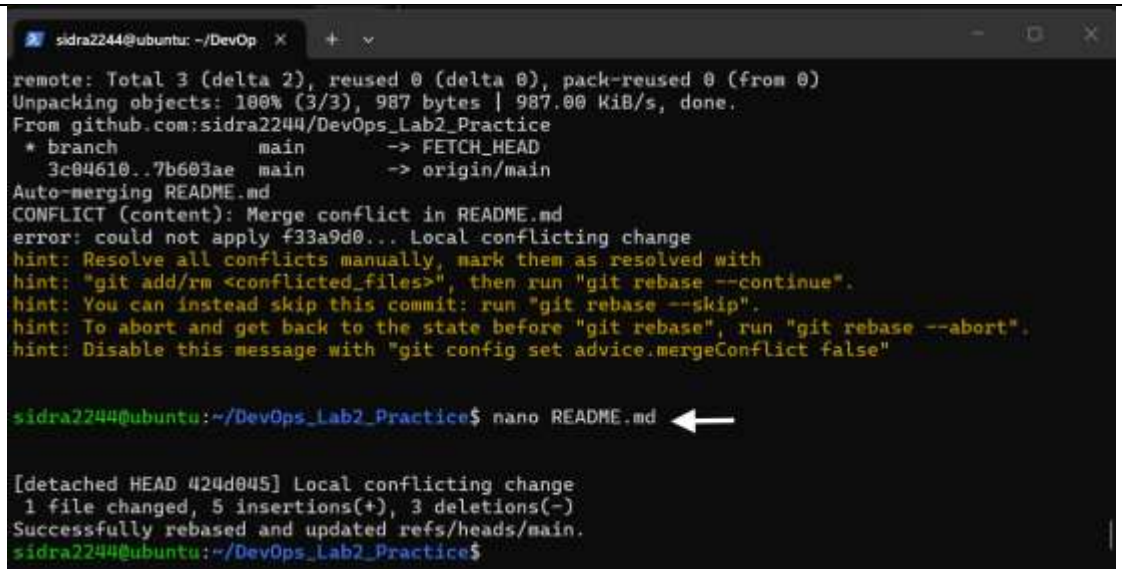
Git reported a conflict during git pull --rebase.

Screenshot: resolved_readme.png



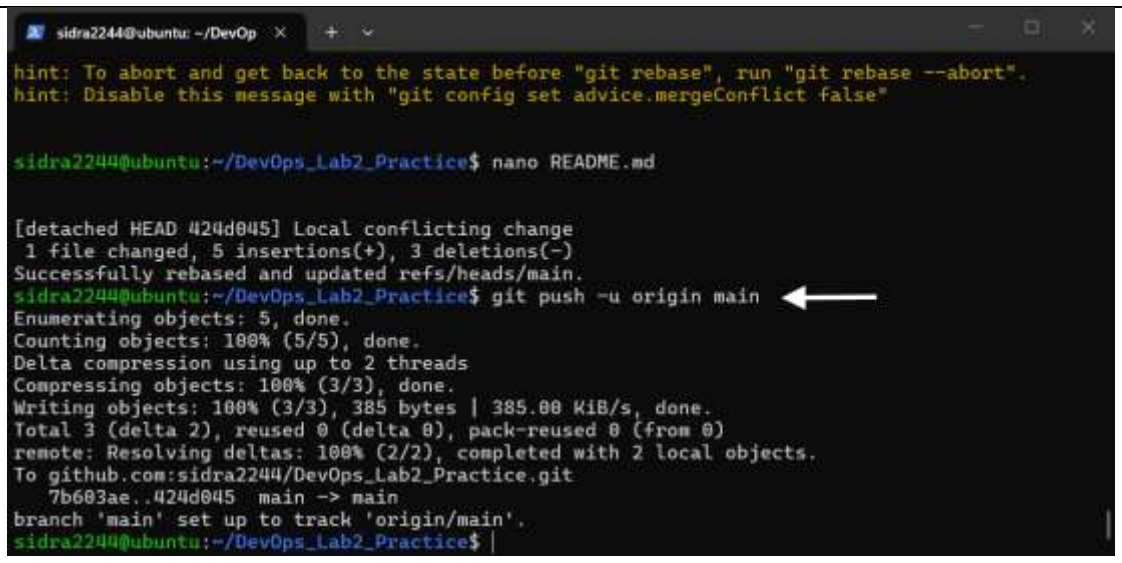
Conflict markers were removed and the intended final text was kept.

Screenshot: rebase_continue.png



The resolved file was staged and the rebase was continued.

Screenshot: push_after_resolve.png



The resolved changes were pushed to GitHub.

Task Summary

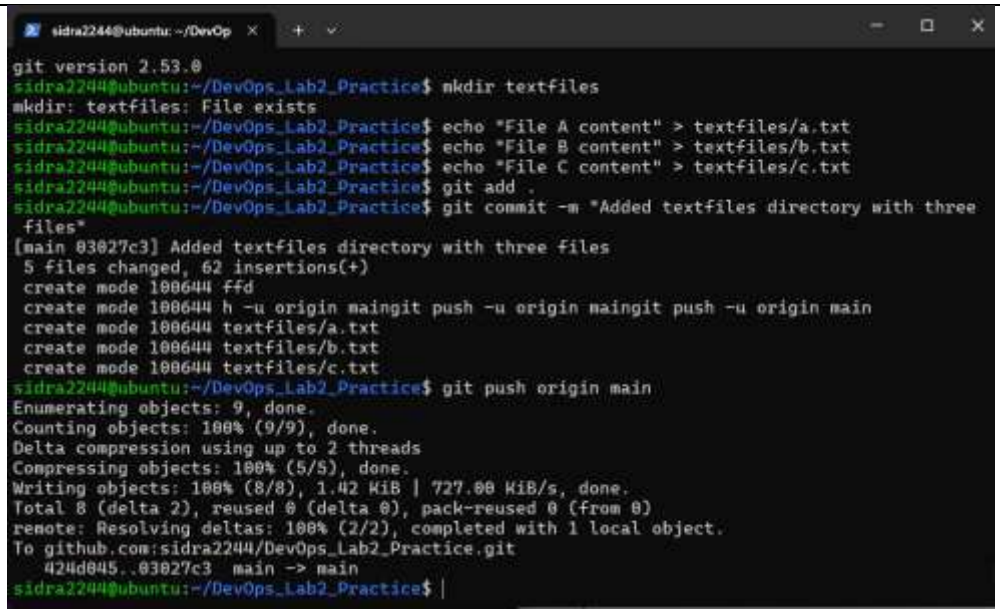
Manual conflict resolution requires choosing the correct final content, staging the file, continuing the rebase, and pushing.

Task 3 – Managing Ignored Files with .gitignore

This task shows that .gitignore prevents matching untracked files from being added, but does not automatically remove files that Git already tracks.

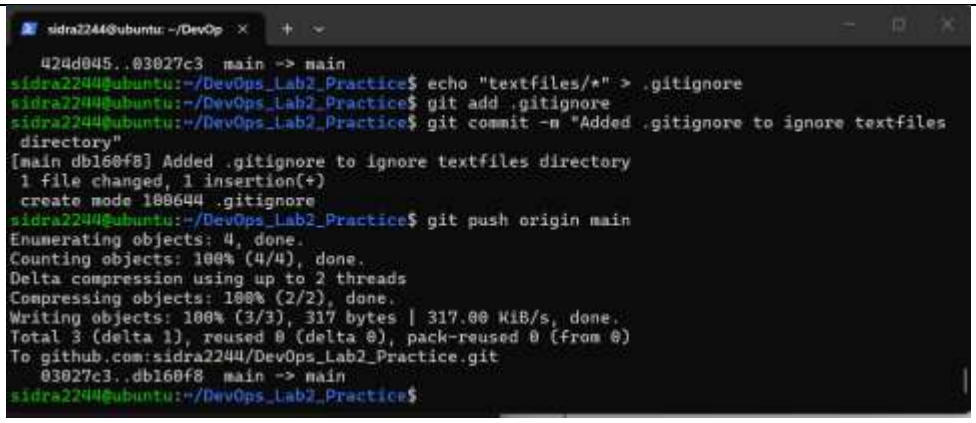
Evidence and Explanation

Screenshot: push_textfiles.png



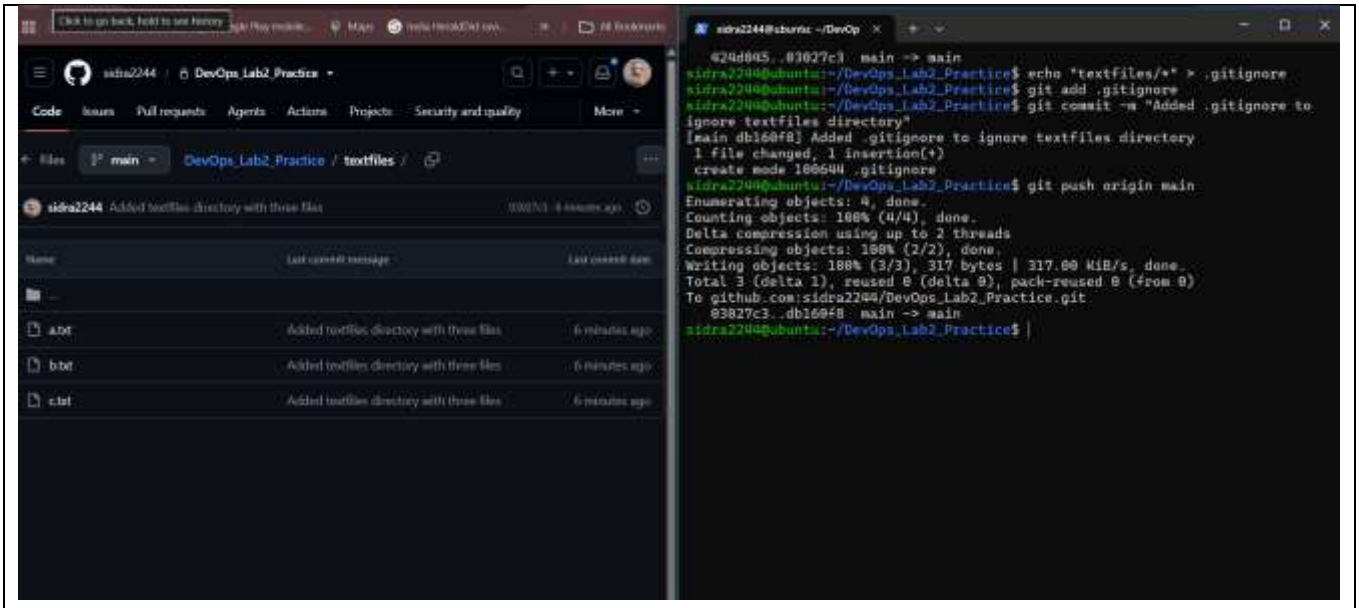
The textfiles directory and its three files were committed and pushed.

Screenshot: gitignore_push.png



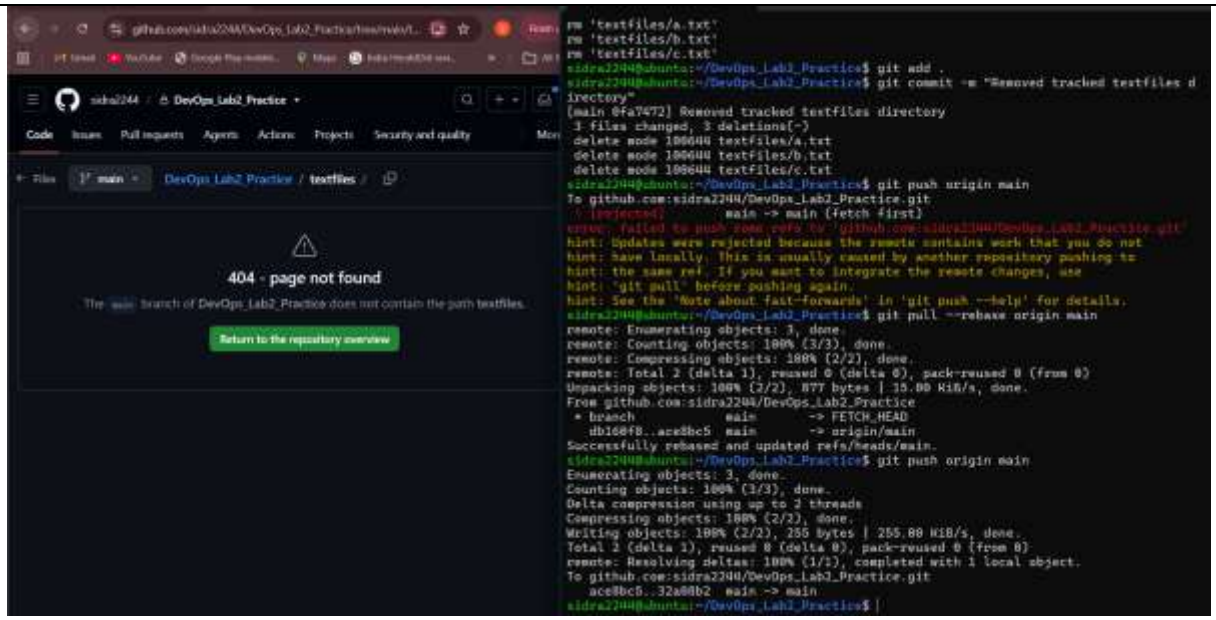
A .gitignore rule was added and pushed.

Screenshot: repo_still_has_textfiles.png



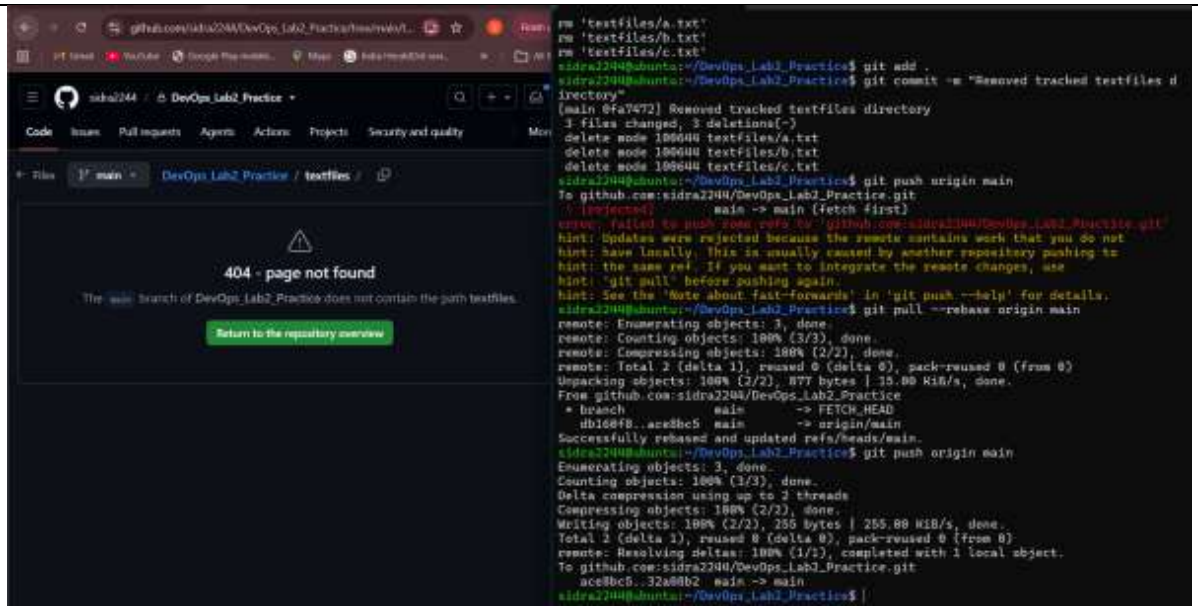
The directory remained visible on GitHub because it was already tracked.

Screenshot: rm_cached_push.png



git rm -r --cached removed the directory from Git tracking, followed by a commit and push.

Screenshot: repo_textfiles_removed.png



The directory was no longer tracked in the remote repository.

Task Summary

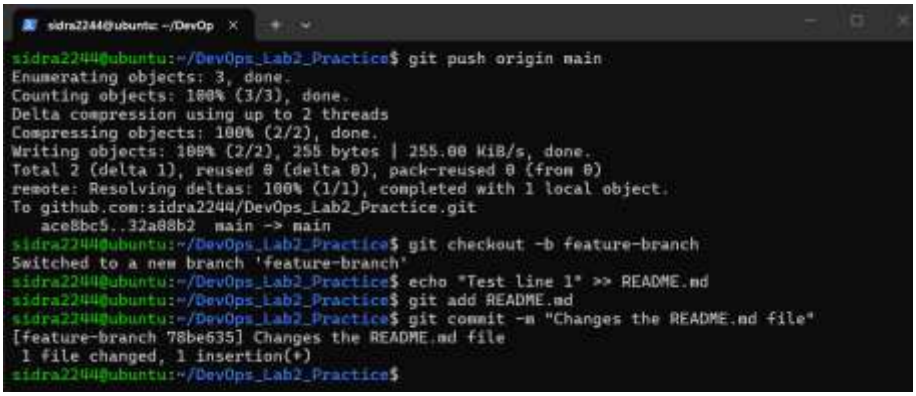
Use .gitignore for future exclusions and git rm --cached to untrack files already committed while keeping local copies.

Task 4 – Temporary Changes with git stash

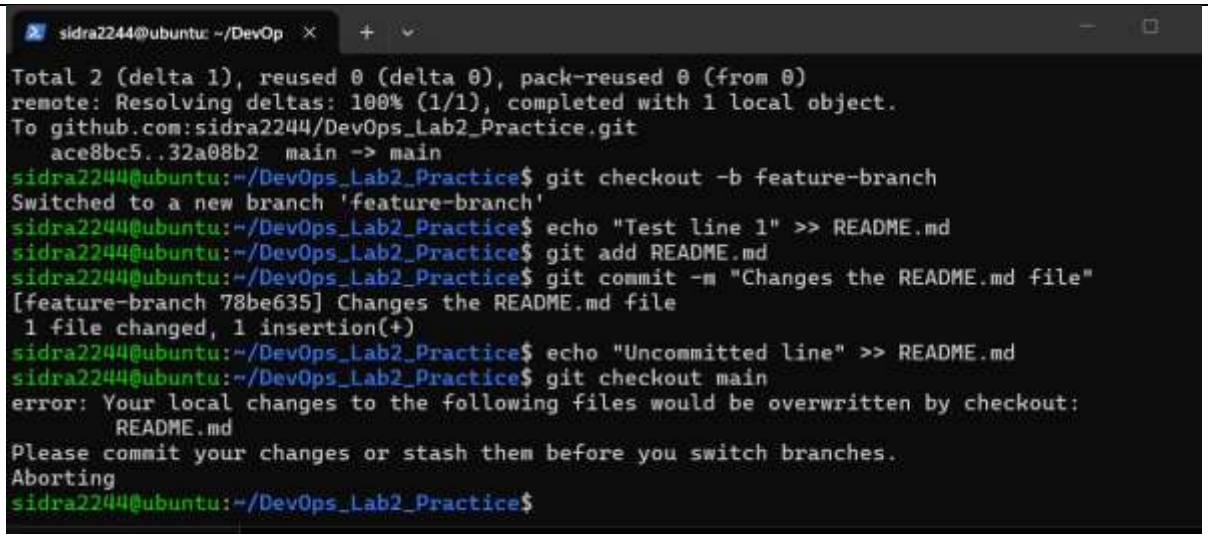
This task uses git stash to save uncommitted work temporarily, allowing a branch switch without losing the edits.

Evidence and Explanation

Screenshot: modified_readme.png

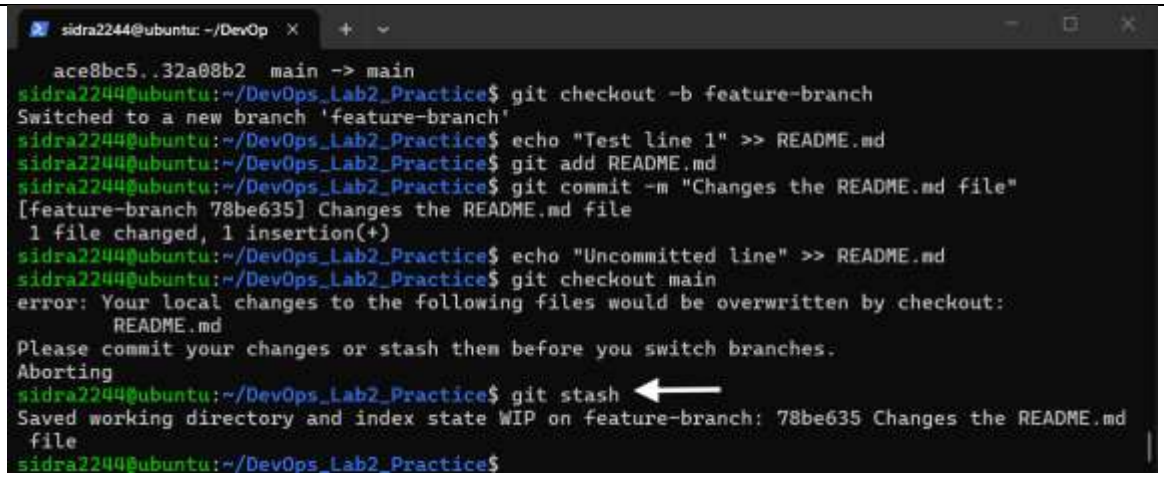


Screenshot: checkout_error.png



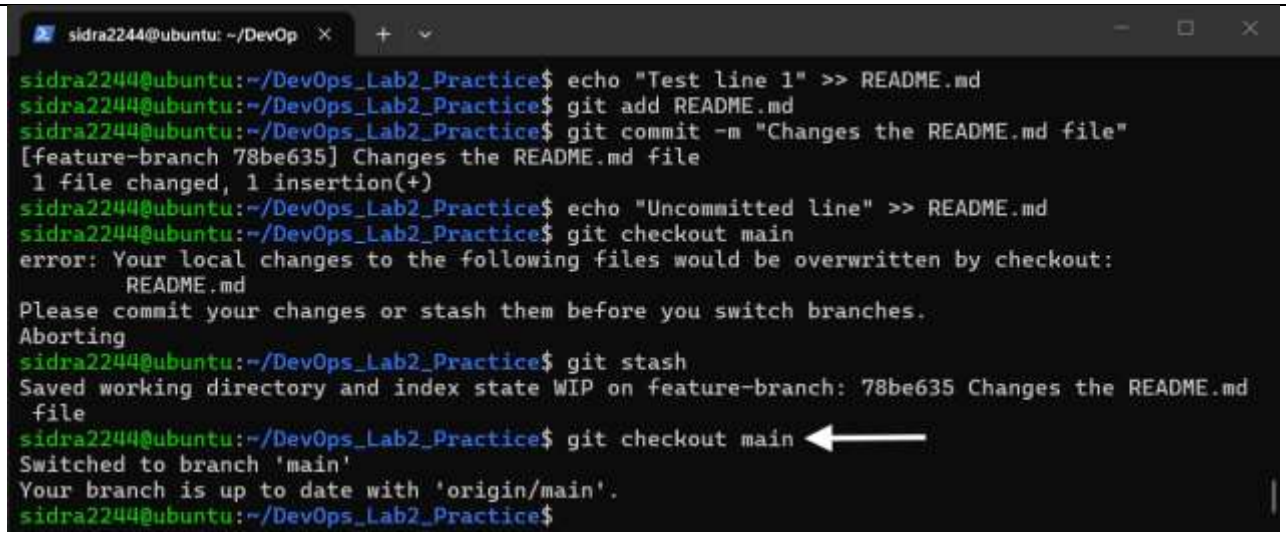
Git prevented switching branches because uncommitted changes could be overwritten.

Screenshot: stash_command.png



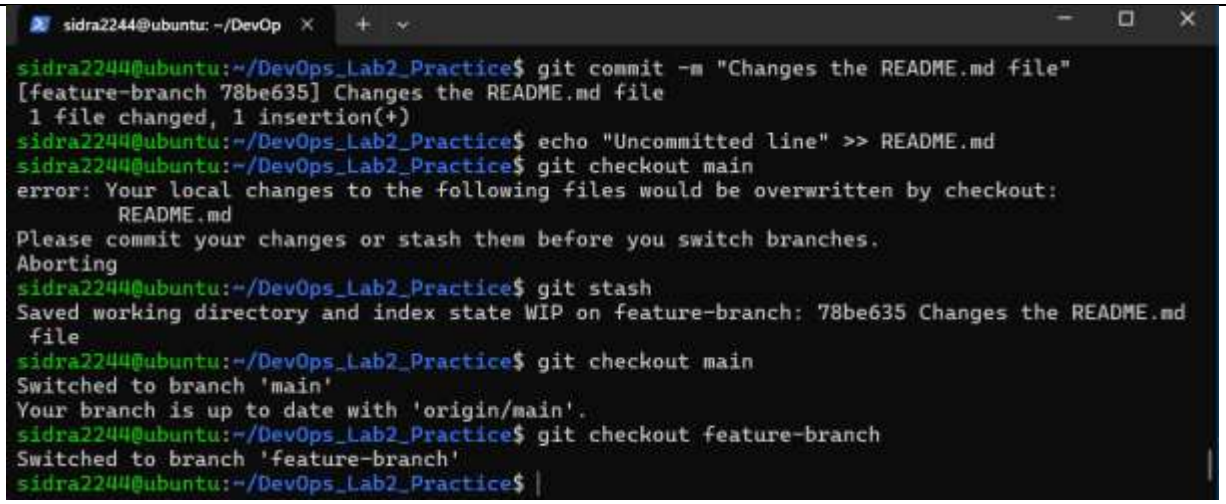
git stash saved the uncommitted changes temporarily.

Screenshot: branch_switched.png



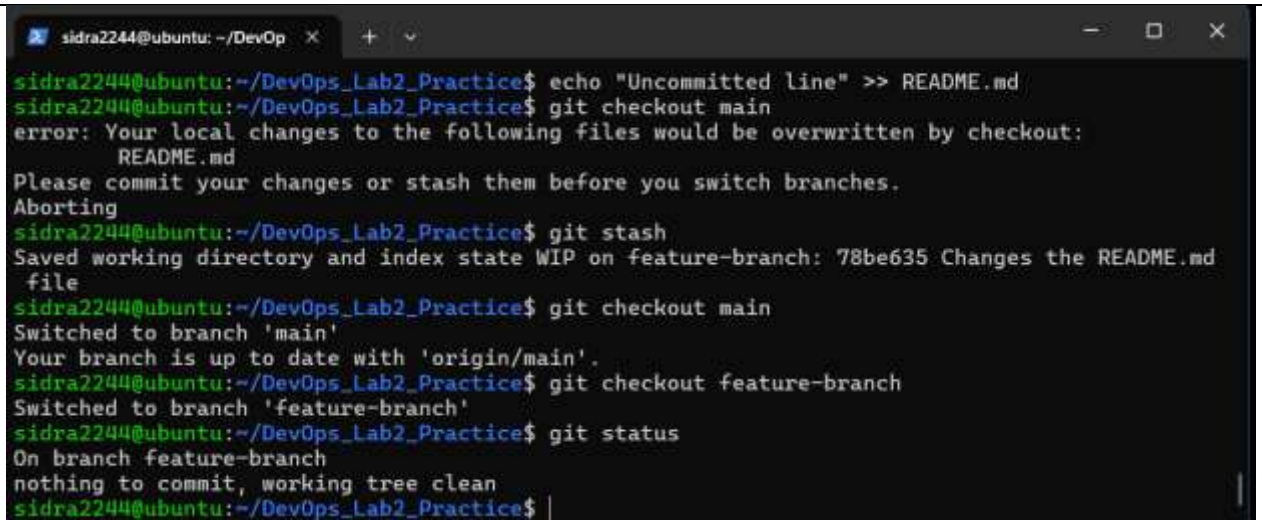
After stashing, switching to main succeeded.

Screenshot: back_to_feature.png



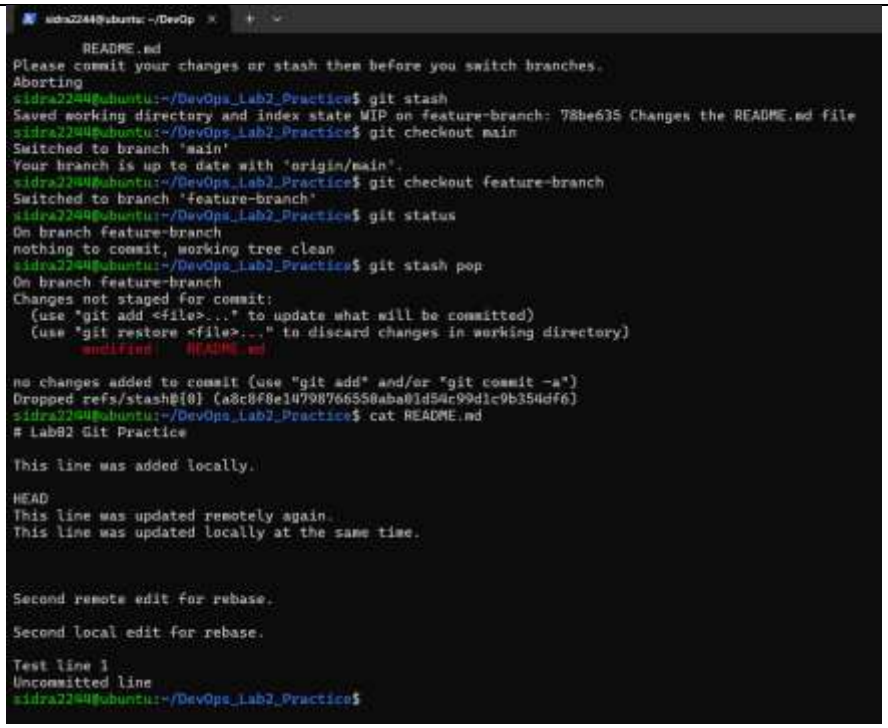
The feature branch was checked out again.

Screenshot: status_clean.png



The working tree was clean before restoring the stash.

Screenshot: stash_pop.png



git stash pop restored the saved changes.

Task Summary

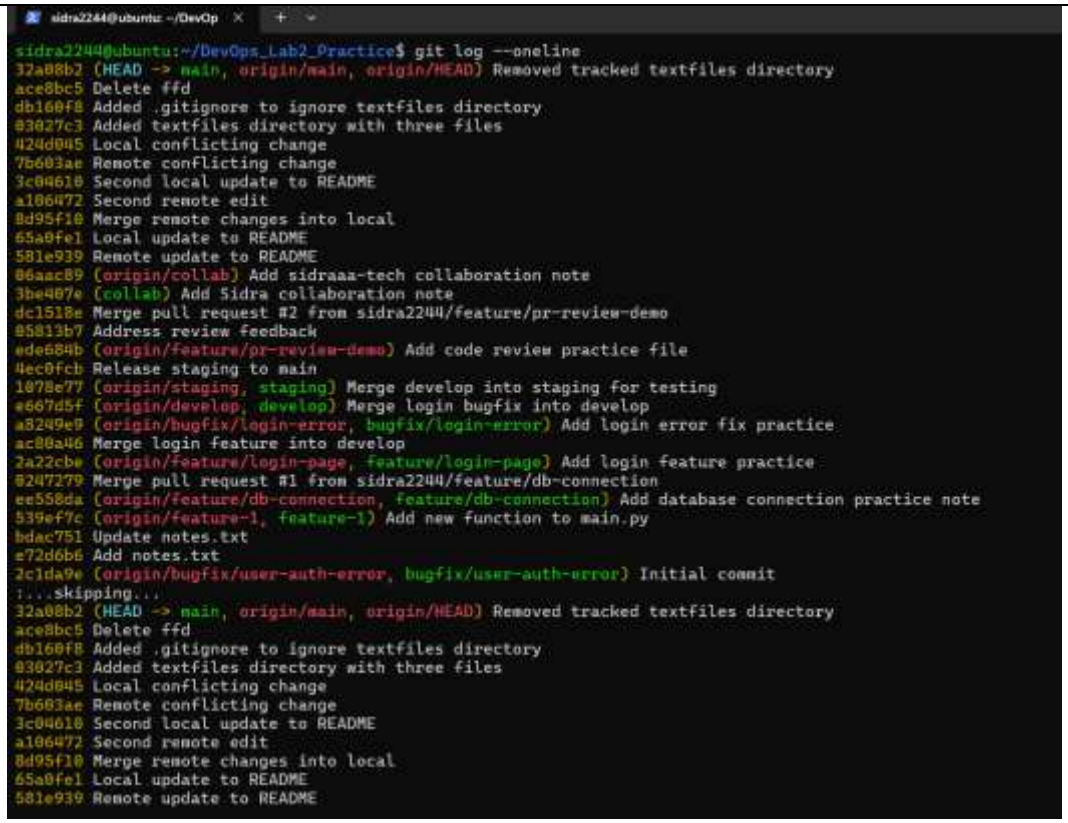
Stashing is useful for temporarily setting aside work. git stash pop reapplies the saved changes.

Task 5 – Checking Out a Specific Commit

This task inspects commit history and checks out an earlier commit using its hash.

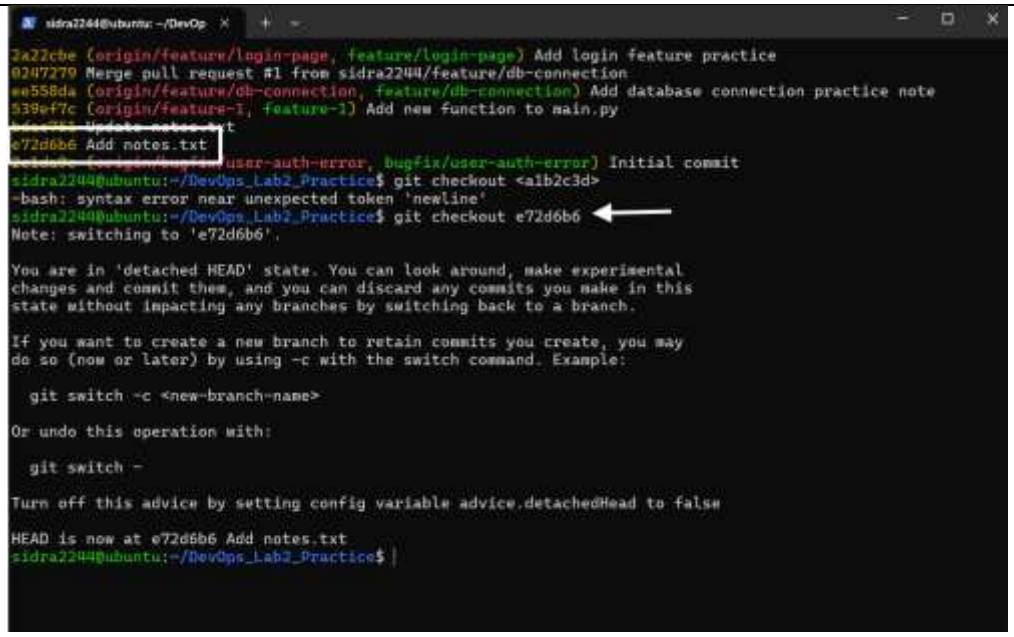
Evidence and Explanation

Screenshot: log_before_checkout.png



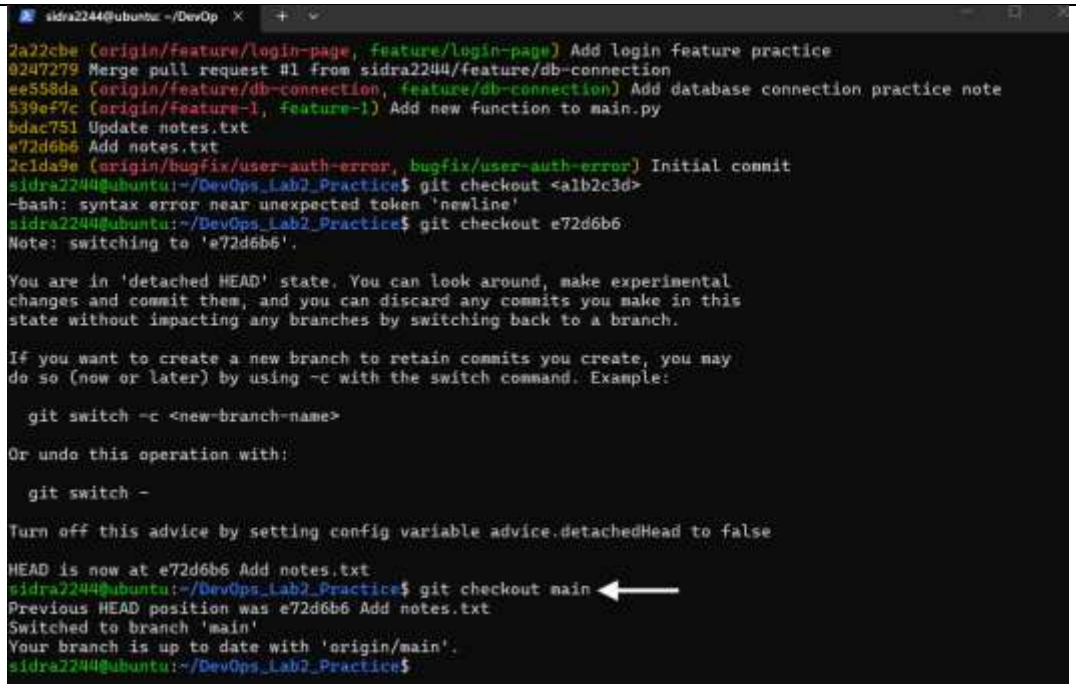
git log --oneline displayed the recent commit history.

Screenshot: detached_head.png



A specific commit was checked out, placing Git in detached HEAD state.

Screenshot: back_to_main.png



The main branch was checked out again after inspecting the old commit.

Task Summary

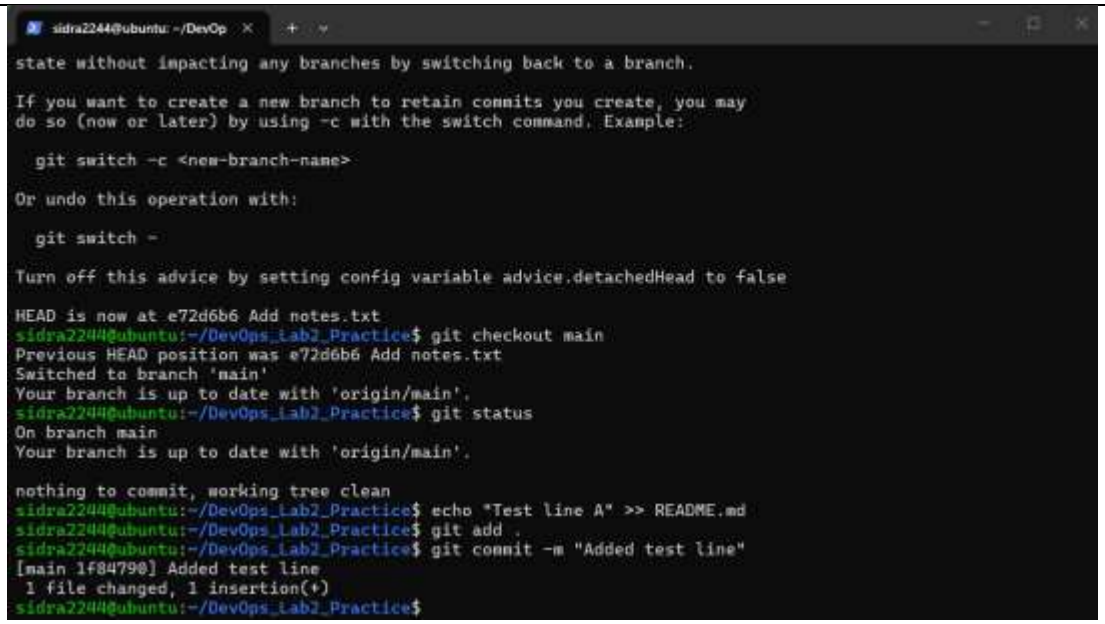
A commit hash can be used to inspect an earlier version. Return to a branch when finished.

Task 6 – Resetting Commits (Soft vs Hard Reset)

This task compares soft reset, which keeps changes staged, with hard reset, which moves the branch pointer and discards changes in the working tree.

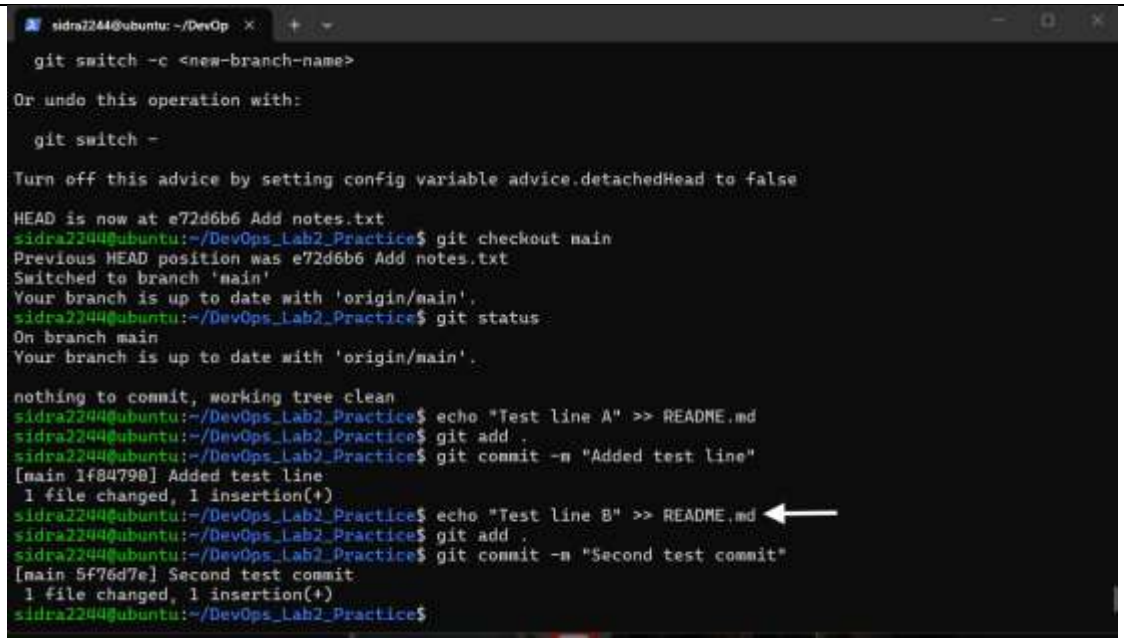
Evidence and Explanation

Screenshot: first_commit.png



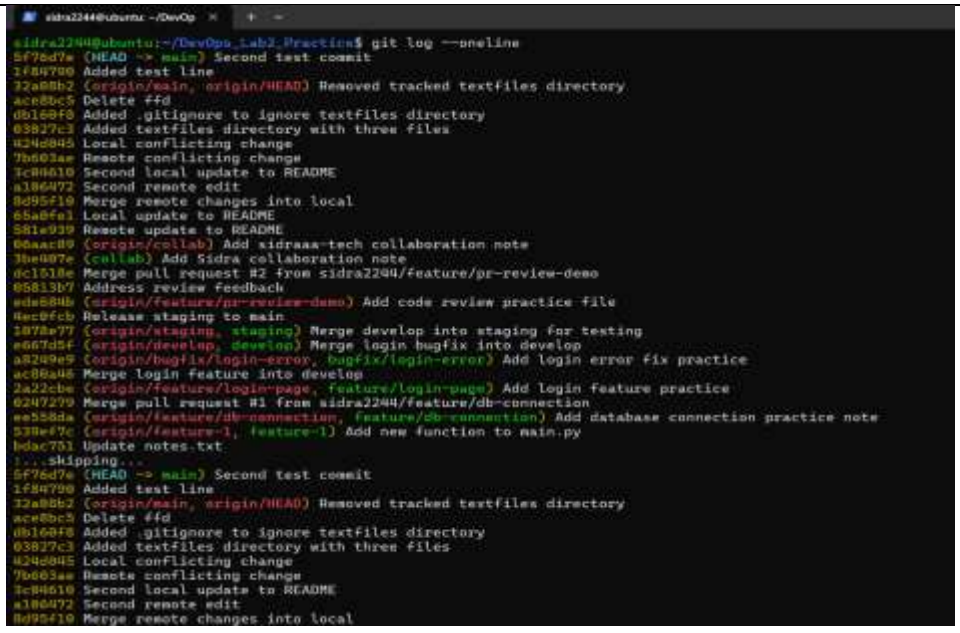
The first test change was committed.

Screenshot: second_commit.png



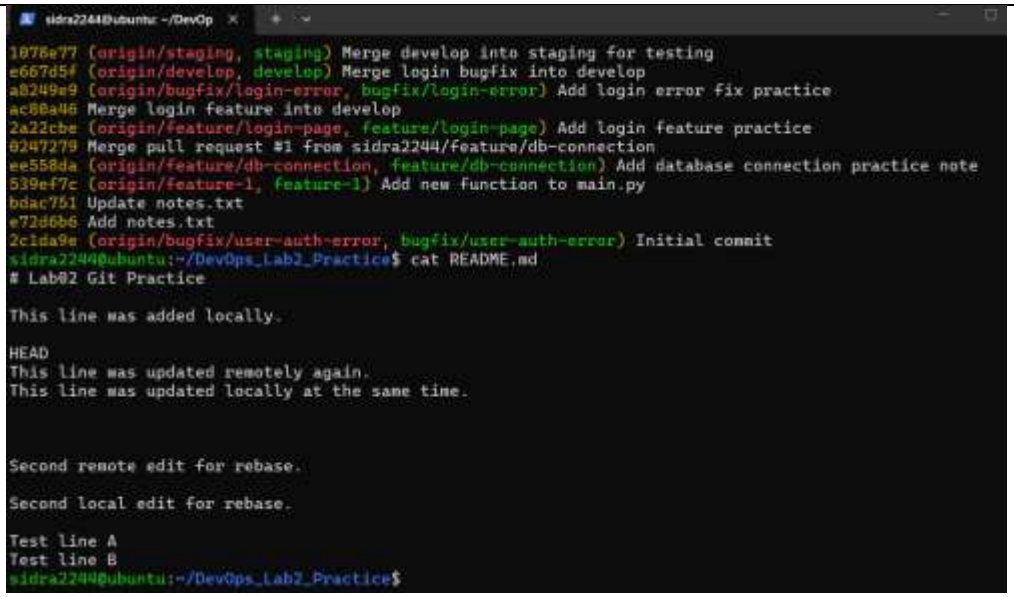
A second test change was committed.

Screenshot: log_before_reset.png



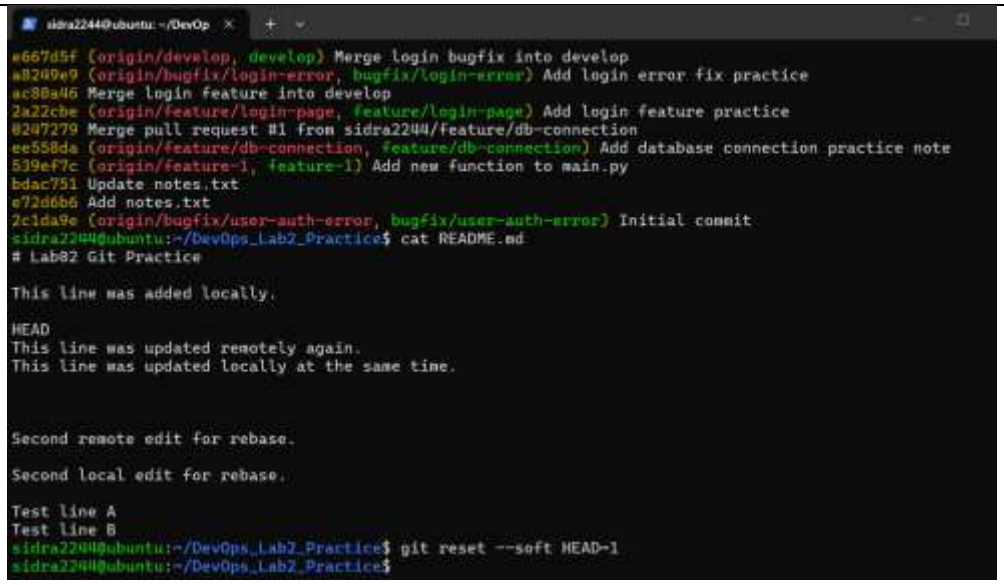
The commit history was checked before resetting.

Screenshot: file_before_reset.png



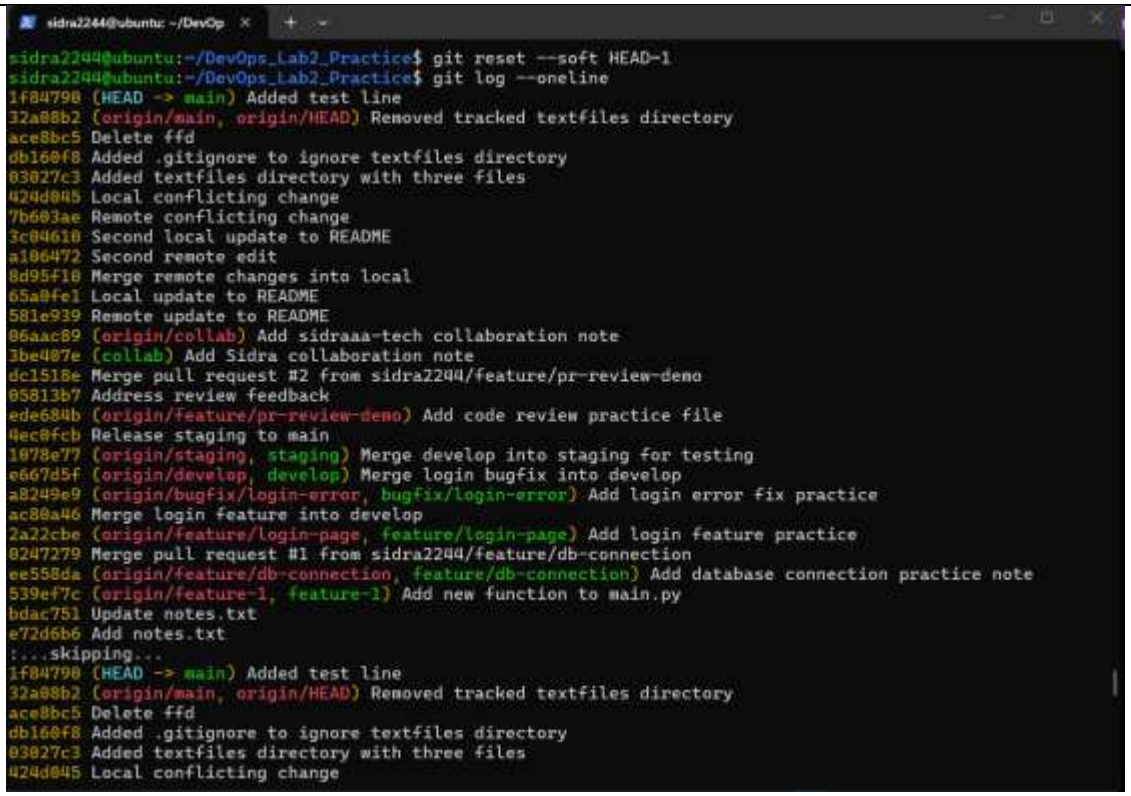
The file showed both test changes before reset.

Screenshot: soft_reset.png



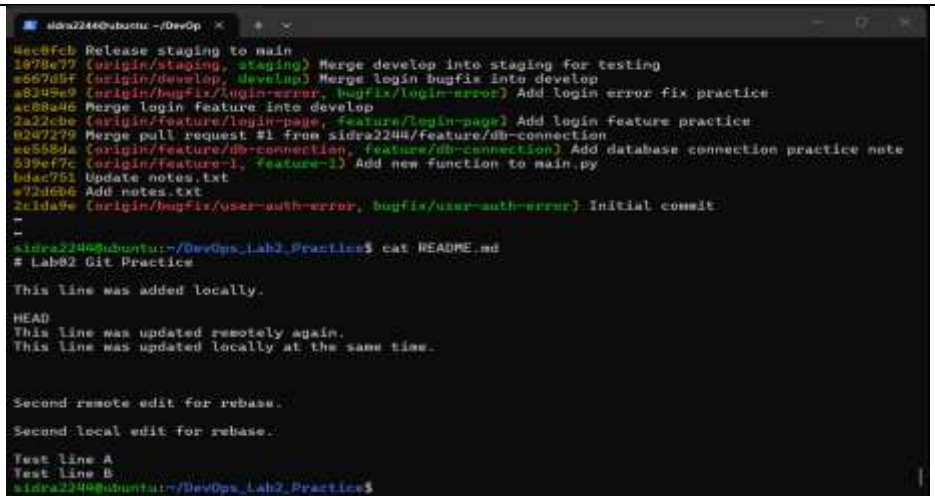
git reset --soft HEAD~1 moved HEAD back while keeping changes staged.

Screenshot: log_after_soft_reset.png



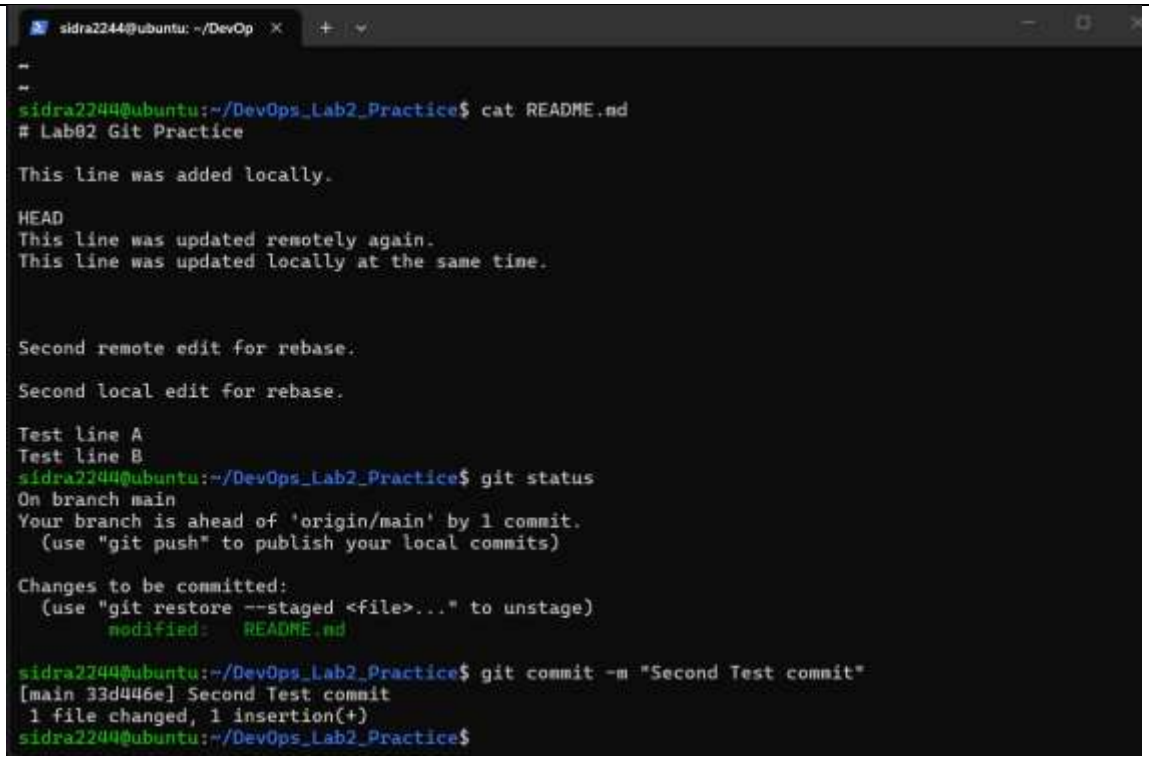
The history was checked after the soft reset.

Screenshot: file_after_soft_reset.png



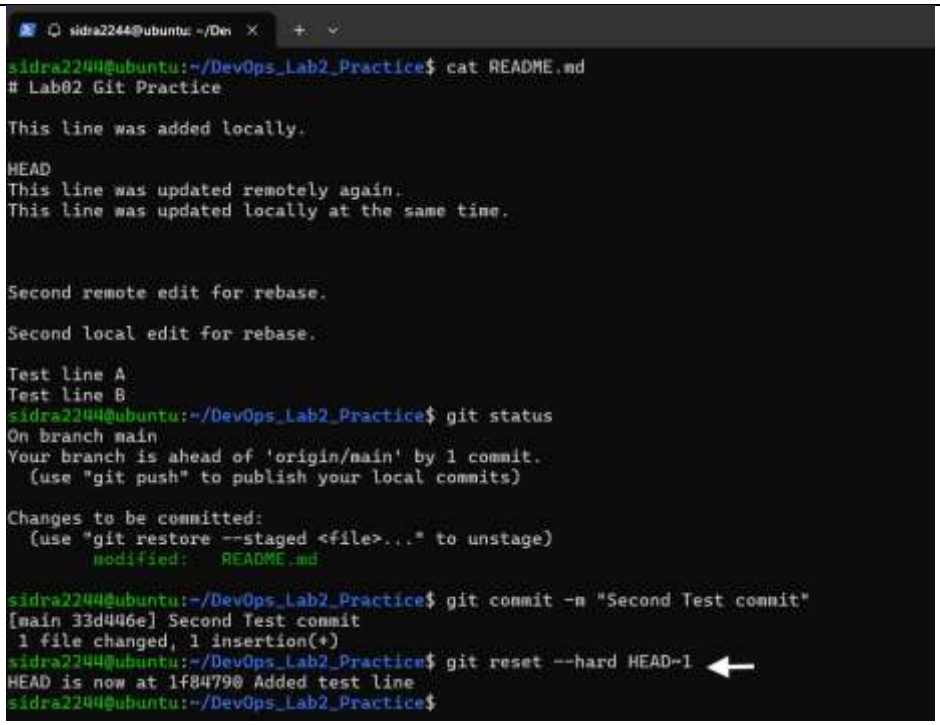
The file contents were verified after the soft reset.

Screenshot: file_after_hard_reset.png



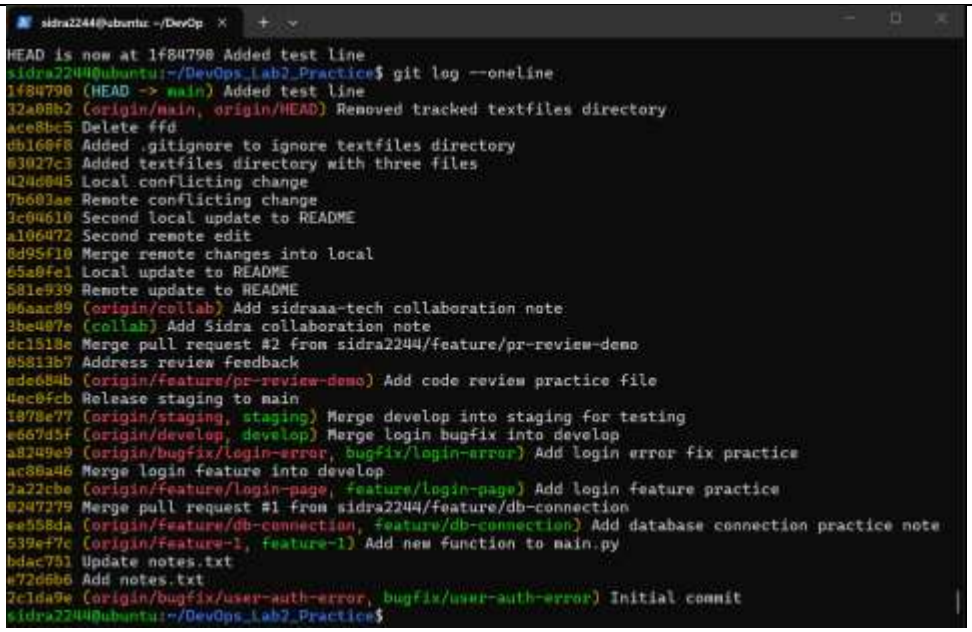
The commit was recreated before performing the hard reset, as required by the lab steps.

Screenshot: hard_reset.png



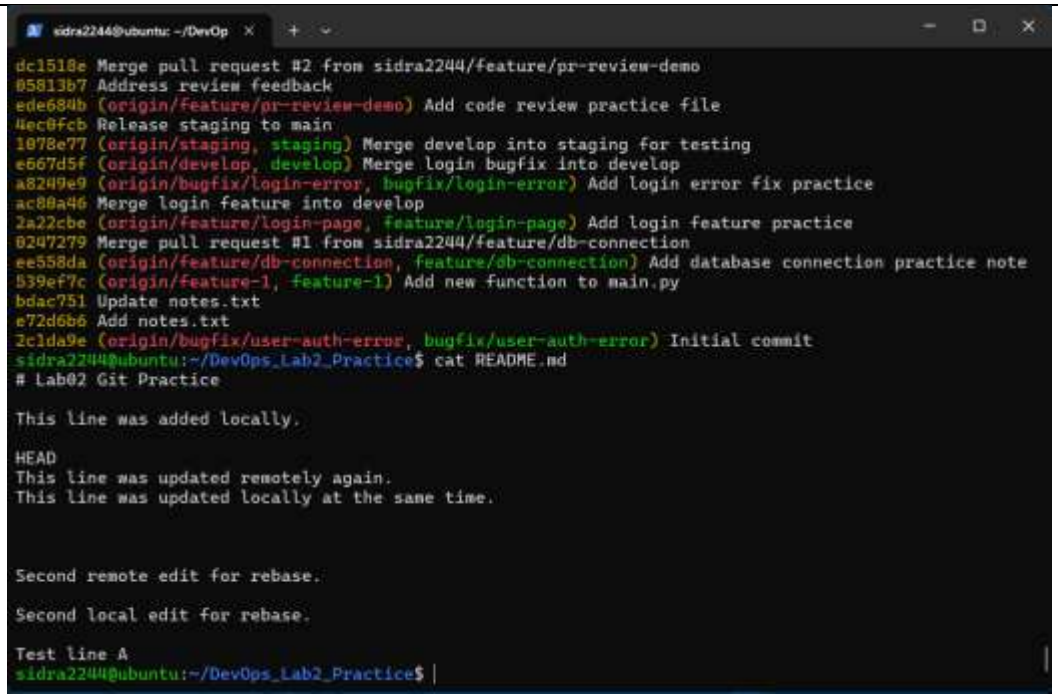
git reset --hard HEAD~1 discarded the latest commit and its associated working-tree changes.

Screenshot: log_after_hard_reset.png



The commit history was checked after the hard reset.

Screenshot: file_after_hard_reset.png



The file was checked to confirm the latest edit was removed.

Task Summary

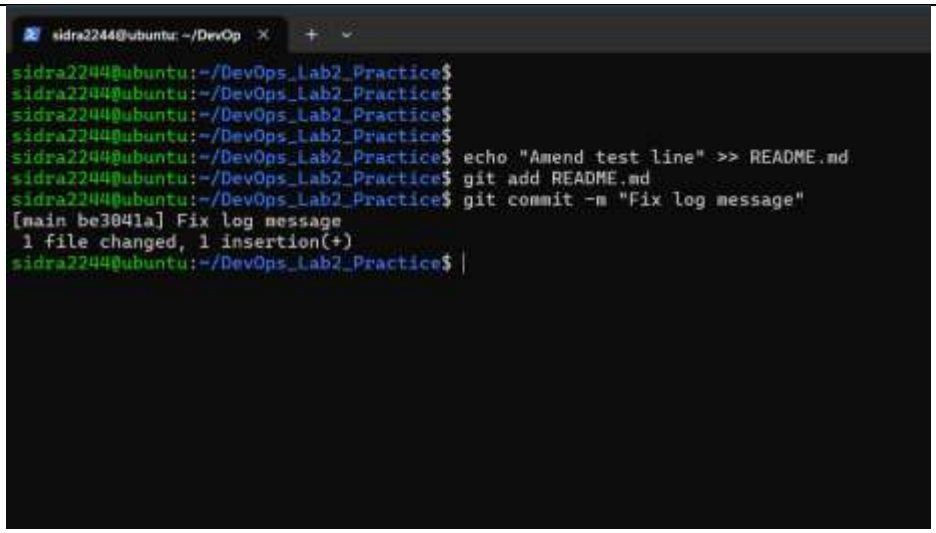
Soft reset preserves changes for recommitting; hard reset discards changes. Use hard reset carefully.

Task 7 – Amending the Last Commit

This task changes the most recent commit by adding a missed file or updating the commit message.

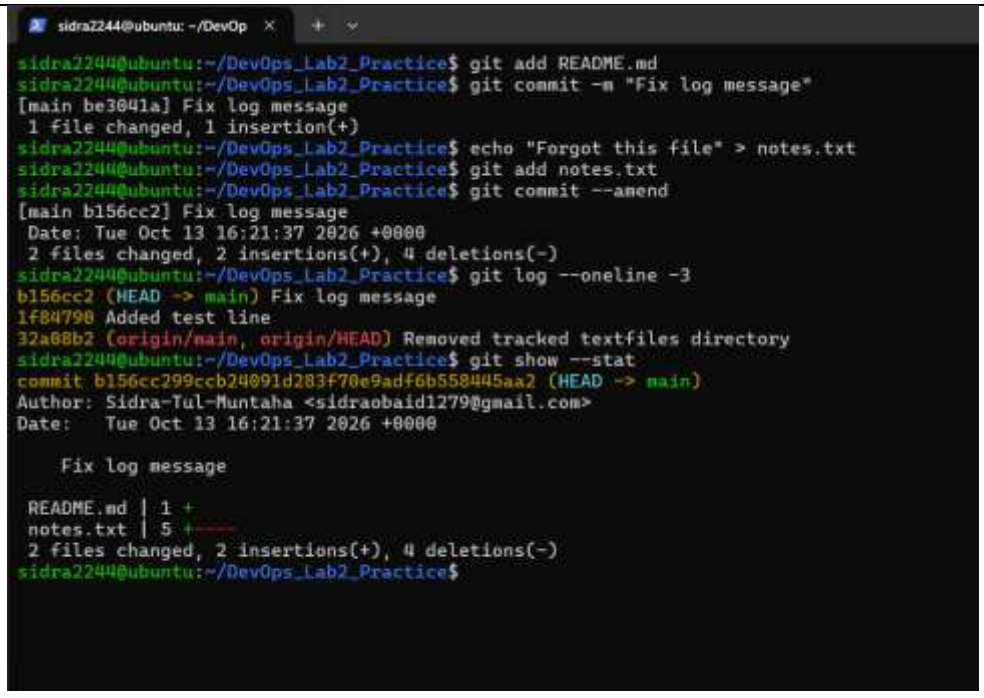
Evidence and Explanation

Screenshot: first_amend_commit.png



The initial change was staged and committed.

Screenshot: amend_commit.png



git commit --amend updated the latest commit with the additional staged change.

Task Summary

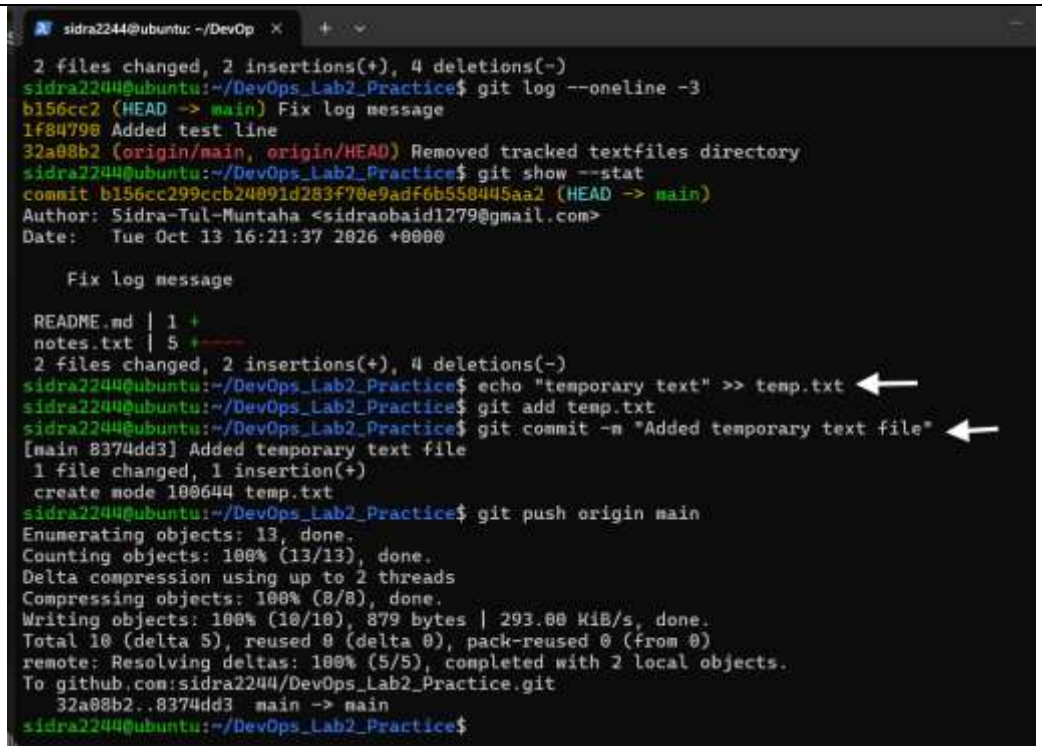
git commit --amend is useful for correcting the most recent commit before sharing it.

Task 8 – Reverting a Commit Safely

This task creates a commit and then uses git revert to undo its changes without deleting the original commit from history.

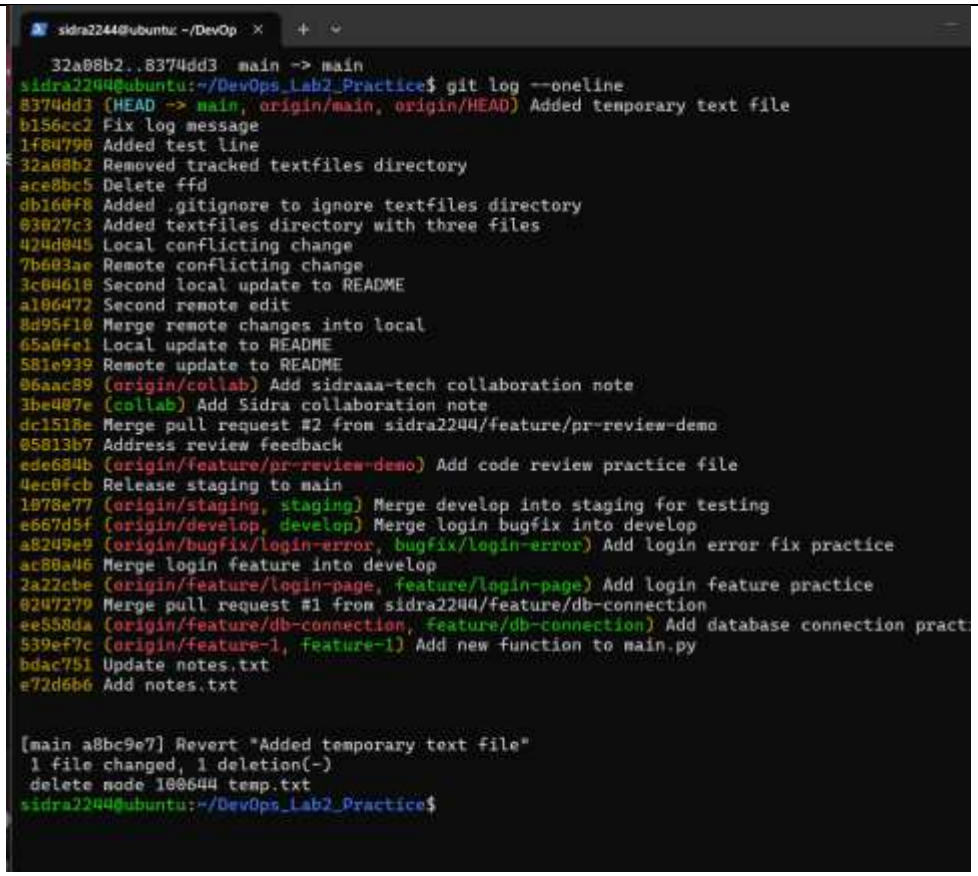
Evidence and Explanation

Screenshot: commit_temp_file.png



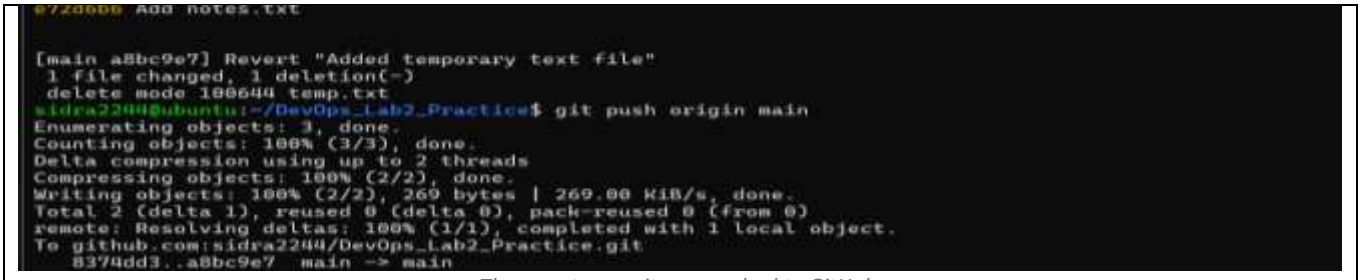
A temporary text change was committed and pushed.

Screenshot: revert_commit.png



git revert created a new commit that reverses the selected commit.

Screenshot: revert_push.png



The revert commit was pushed to GitHub.

Task Summary

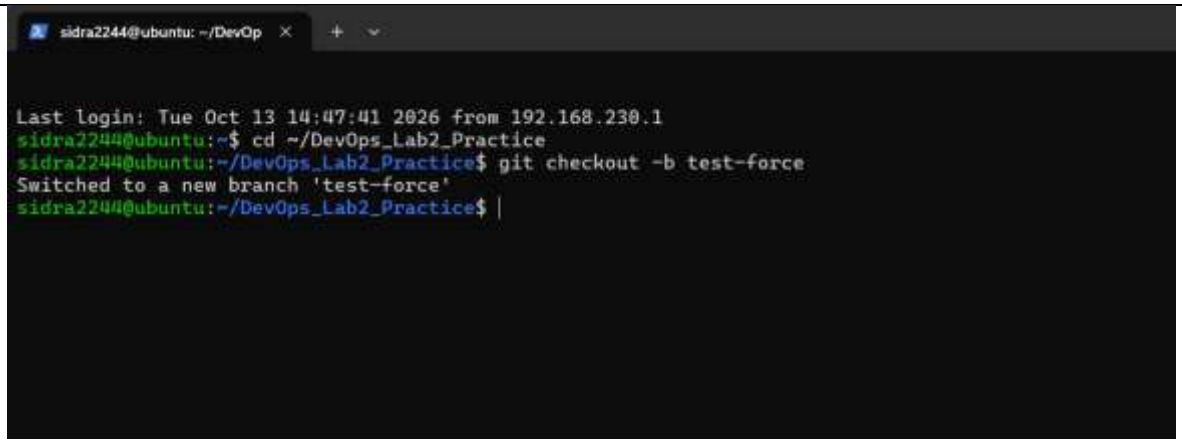
git revert is generally safer than rewriting history on a shared remote branch because it adds a new undo commit.

Task 9 – Force Push with Caution

This task demonstrates a force push on a separate test branch after the branch history is changed with a hard reset.

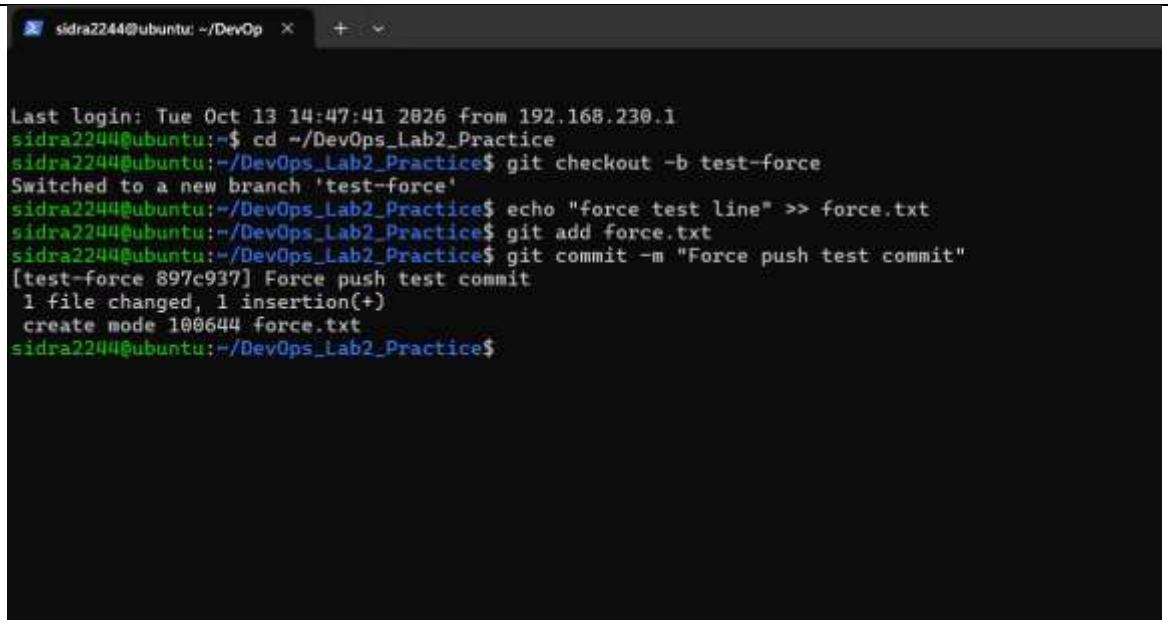
Evidence and Explanation

Screenshot: new_branch.png



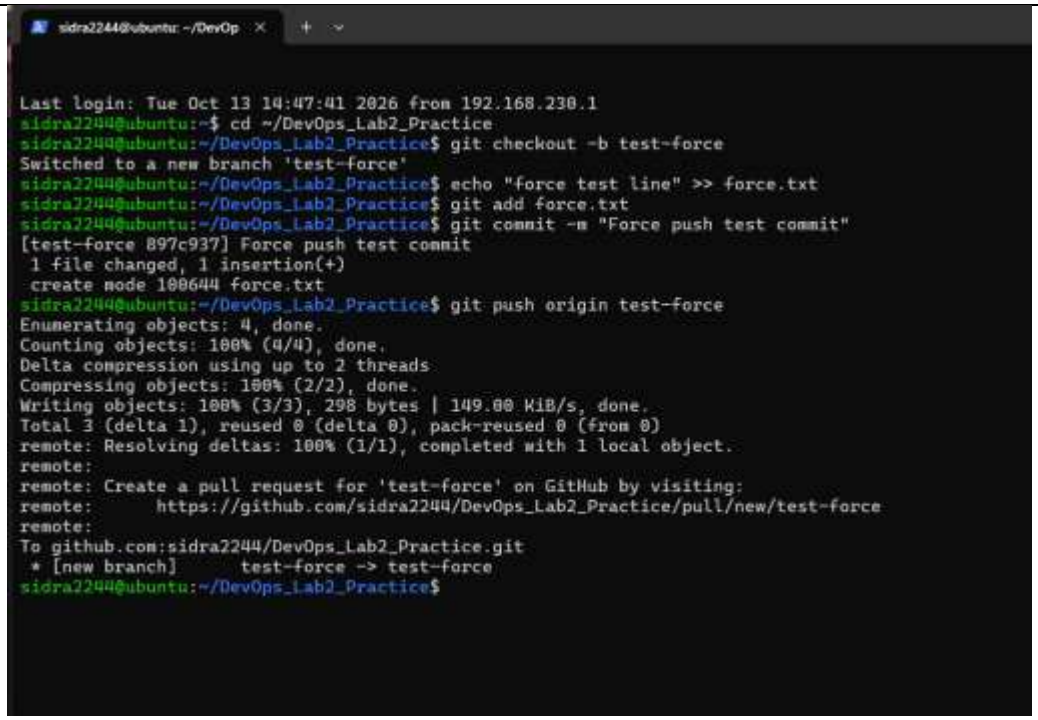
A separate test-force branch was created for this demonstration.

Screenshot: force_commit.png



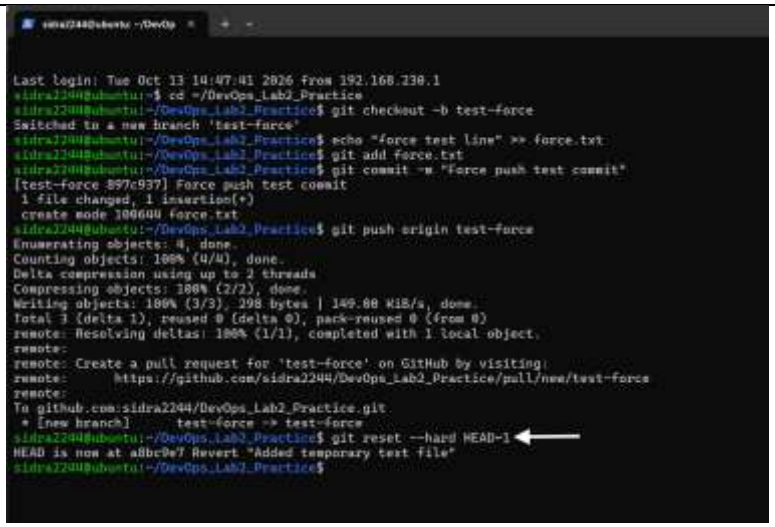
A small change was committed on the test branch.

Screenshot: push_force_branch.png



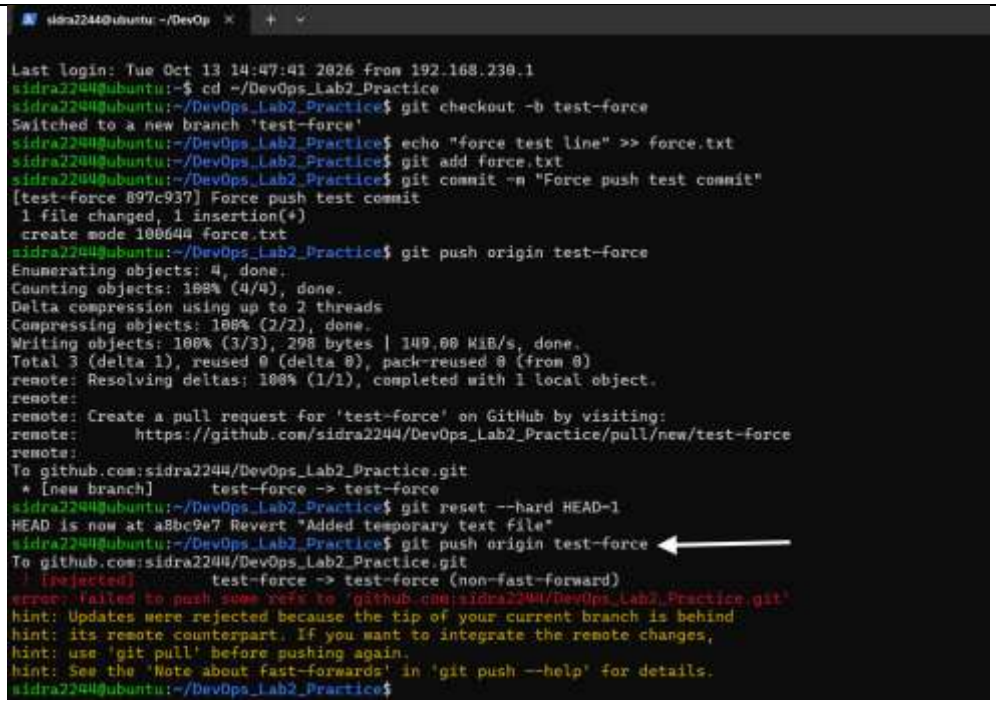
The test branch was pushed to GitHub.

Screenshot: hard_reset_force.png



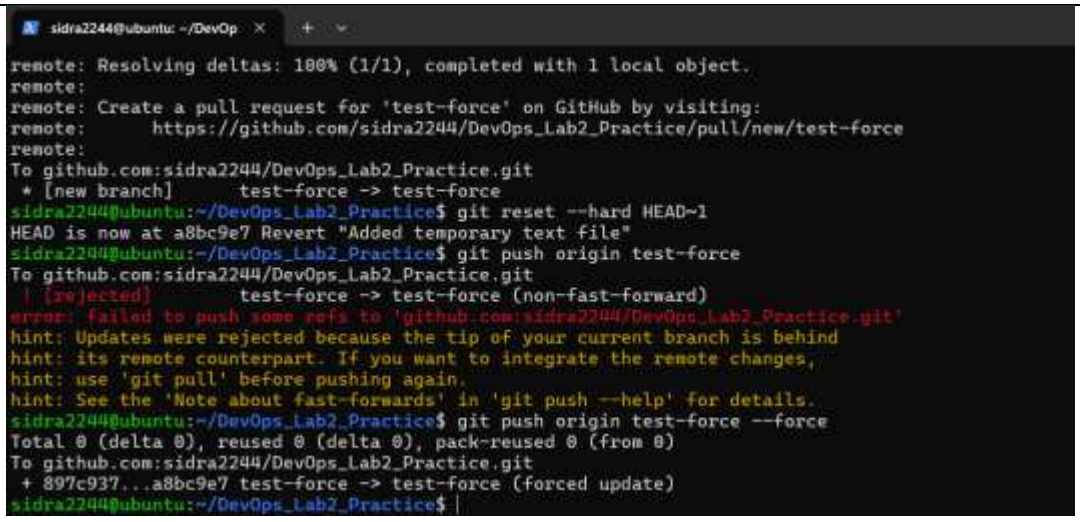
The latest commit was removed locally using a hard reset.

Screenshot: normal_push.png



The normal push was rejected because the remote branch contained the removed commit.

Screenshot: force_push.png



A force push updated the remote branch to match the rewritten local history.

Task Summary

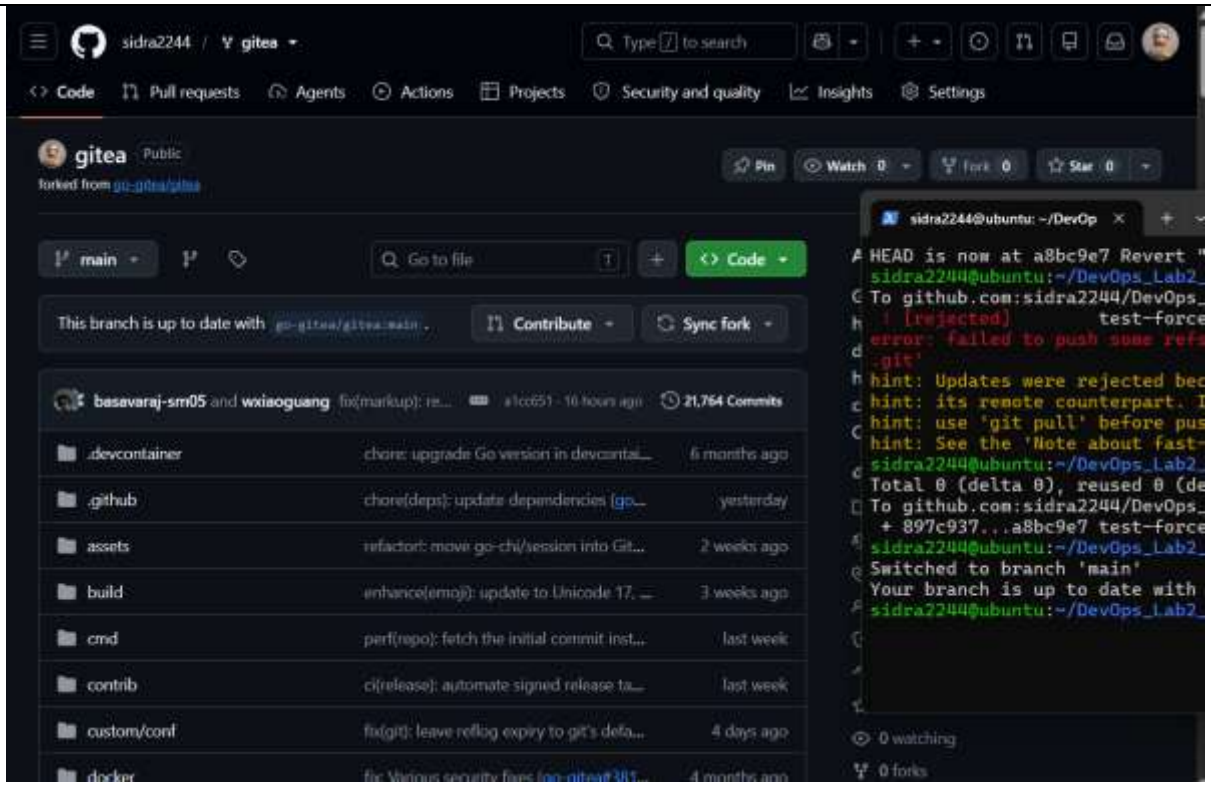
Force push rewrites remote history. It should only be used on a personal test branch when safe.

Task 10 – Running Gitea in GitHub Codespaces via Docker Compose

This task runs the Gitea Git service in a GitHub Codespace using Docker Compose, then accesses its web interface through forwarded port 3000.

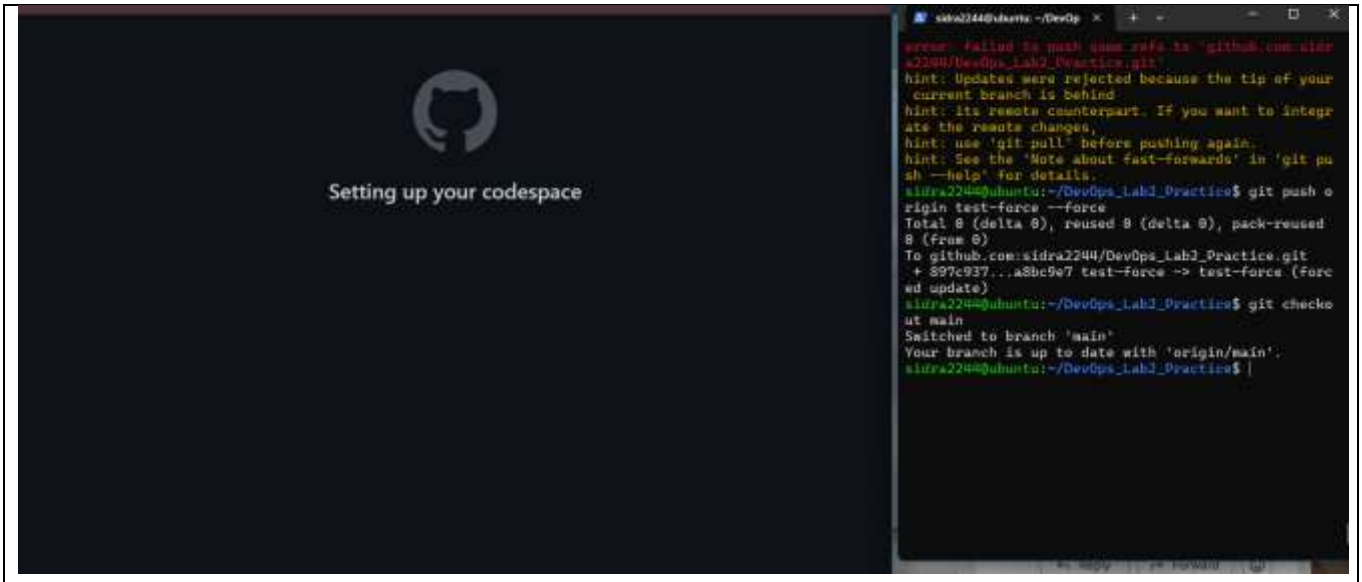
Evidence and Explanation

Screenshot: forked_gitea.png



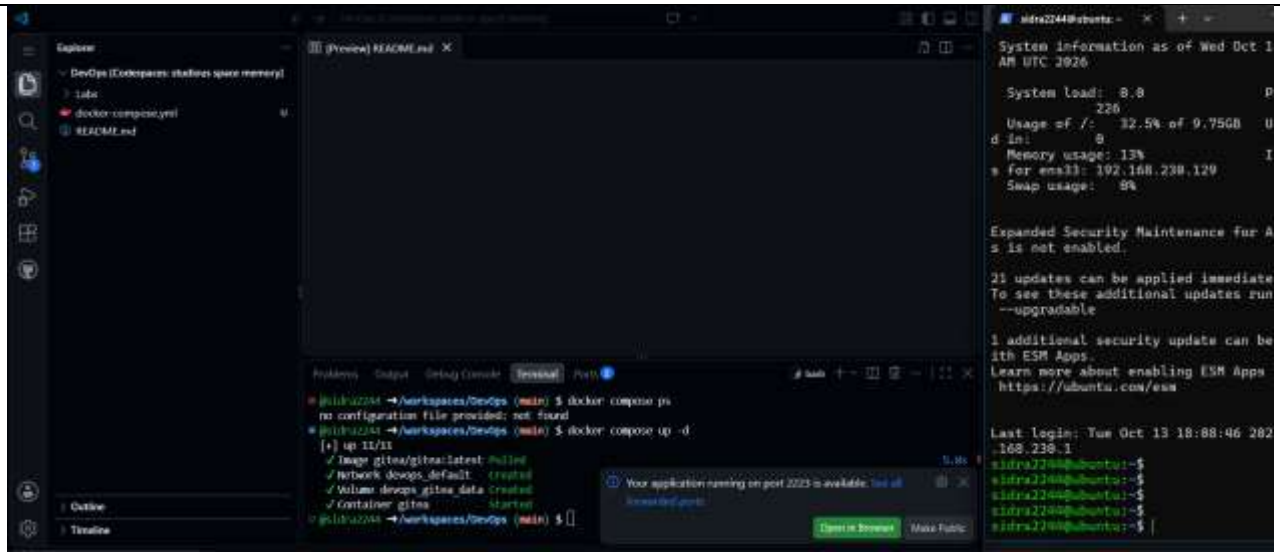
The Gitea repository was forked into the GitHub account.

Screenshot: codespace_loading.png



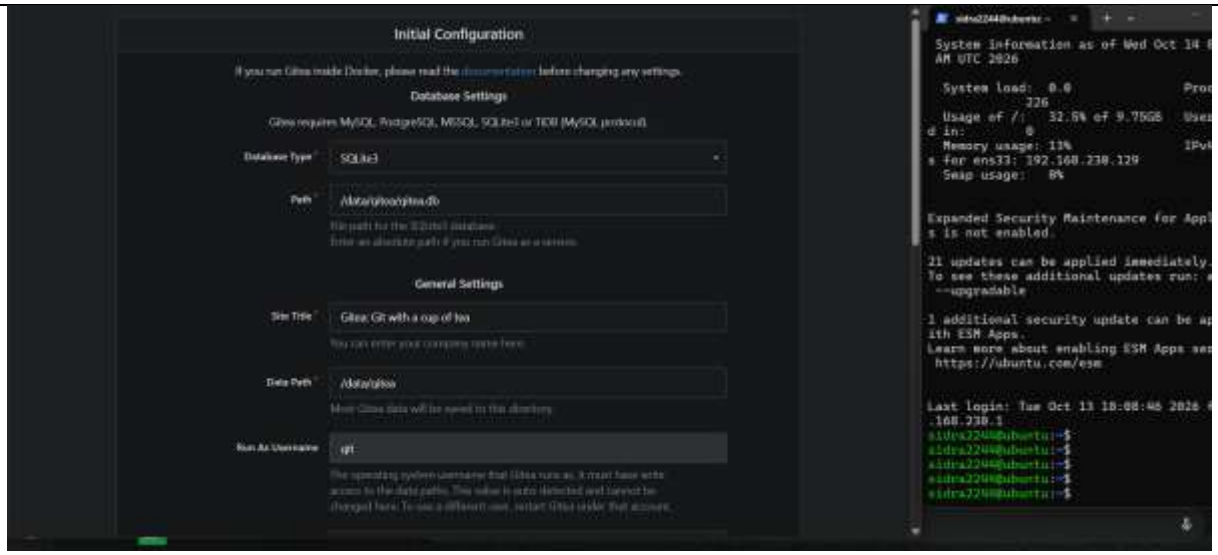
A Codespace was created from the forked repository.

Screenshot: docker_up.png



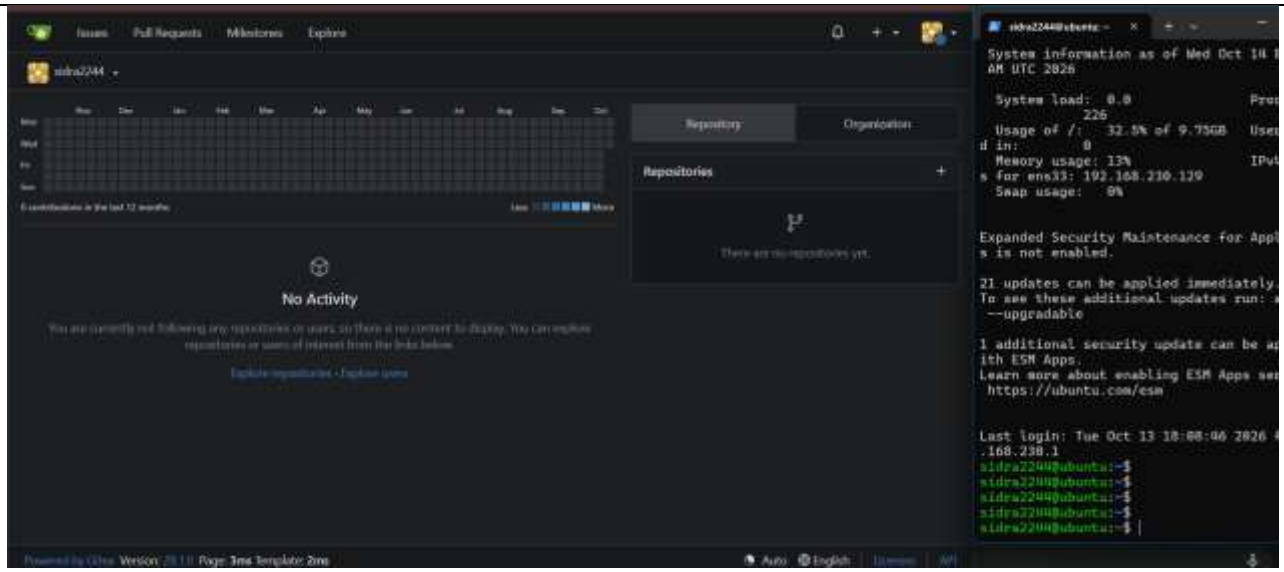
Docker Compose was started and the containers were checked.

Screenshot: gitea_install_page.png



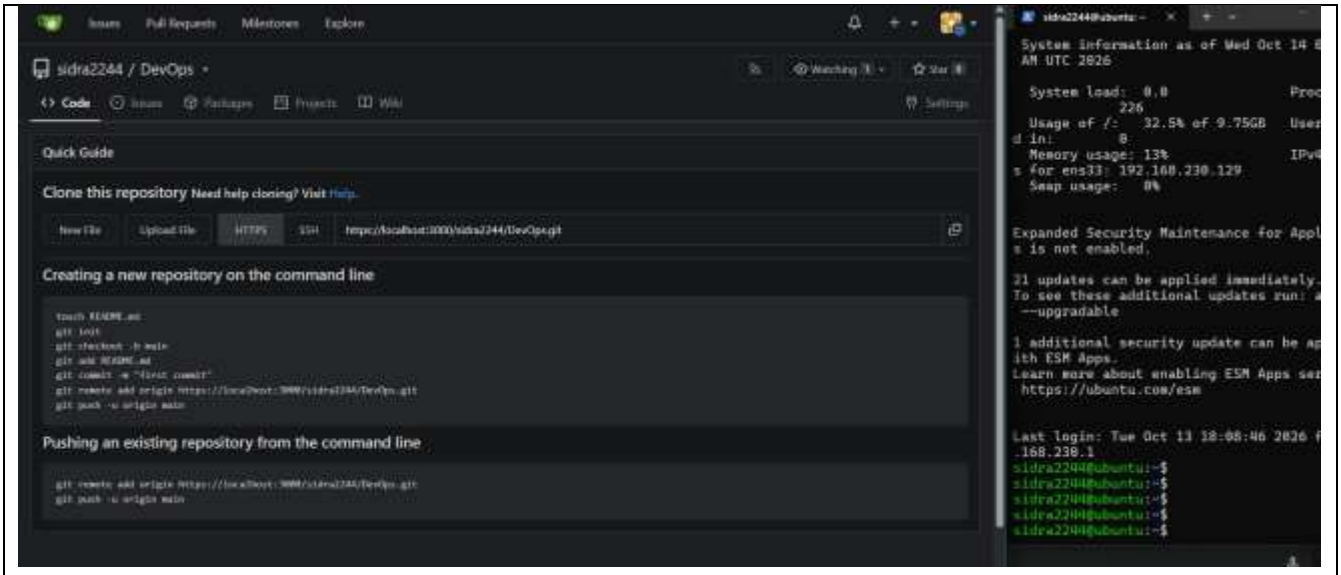
The Gitea installation page was opened through forwarded port 3000.

Screenshot: gitea_dashboard.png



The Gitea dashboard was shown after logging in.

Screenshot: gitea_new_repo.png



A new repository was created in Gitea.

Task Summary

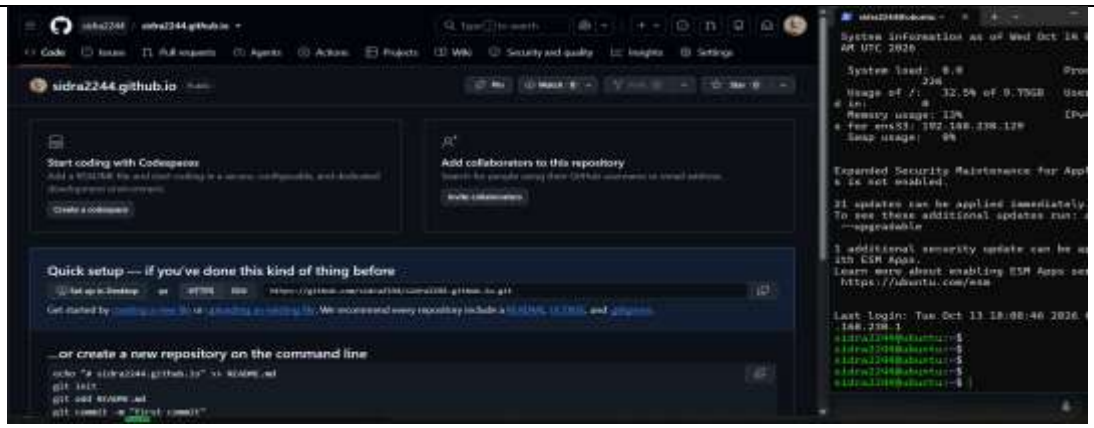
This task demonstrates a self-hosted Git service running in a cloud development environment.

Task 11 – Creating a GitHub Pages Portfolio Site

This task publishes a static portfolio or CV website through a public repository named sidra2244.github.io.

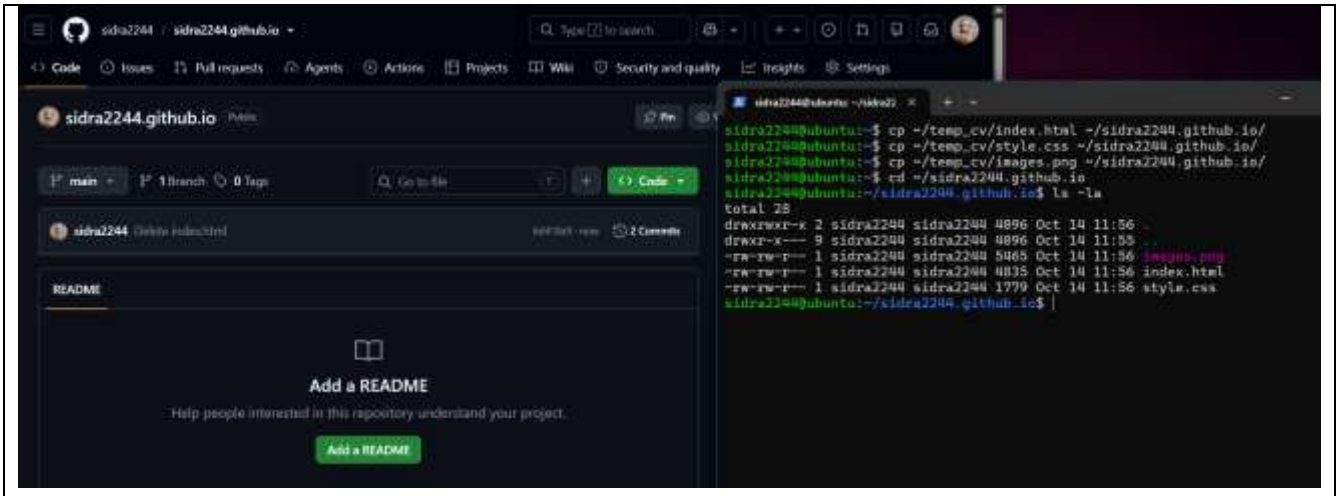
Evidence and Explanation

Screenshot: github_pages_repo.png



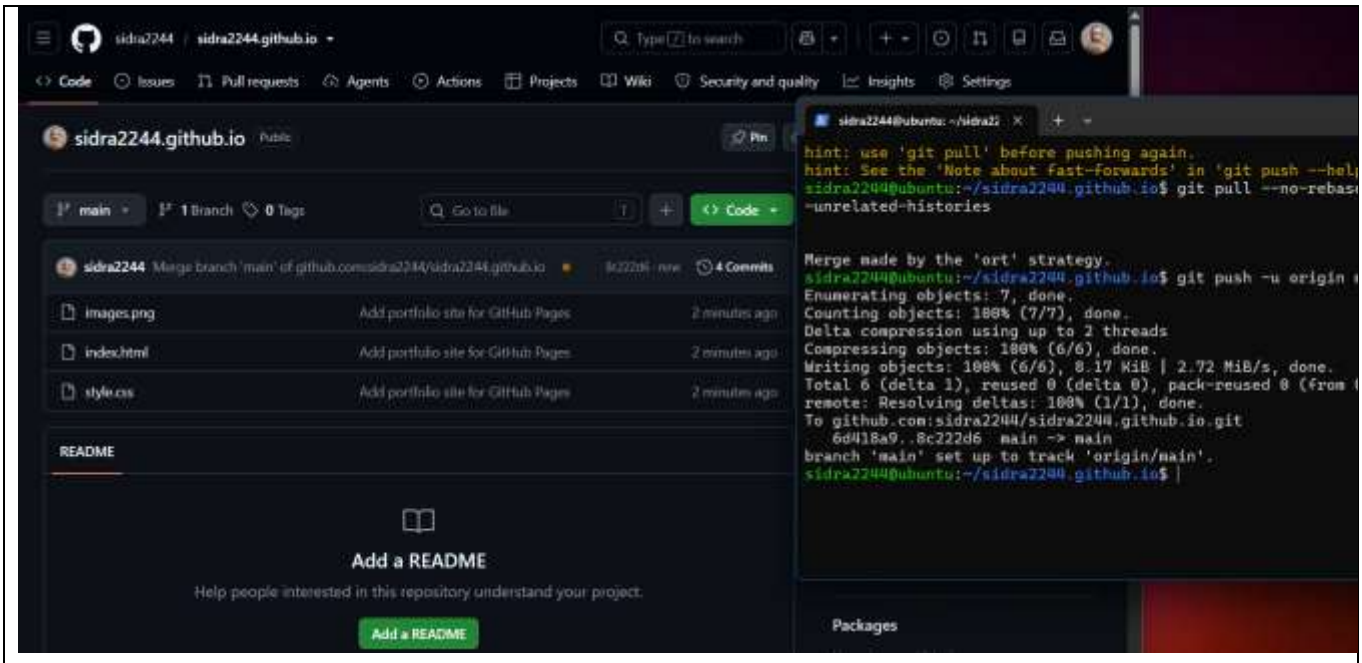
The public GitHub Pages repository was created.

Screenshot: local_static_site.png



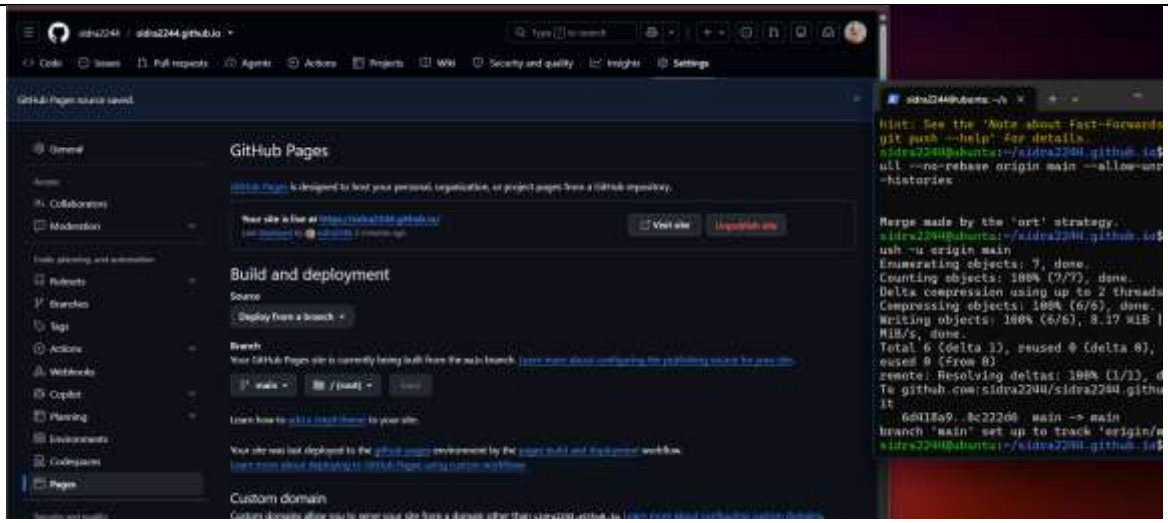
The local HTML, CSS, and JavaScript website files were prepared.

Screenshot: push_static_site.png



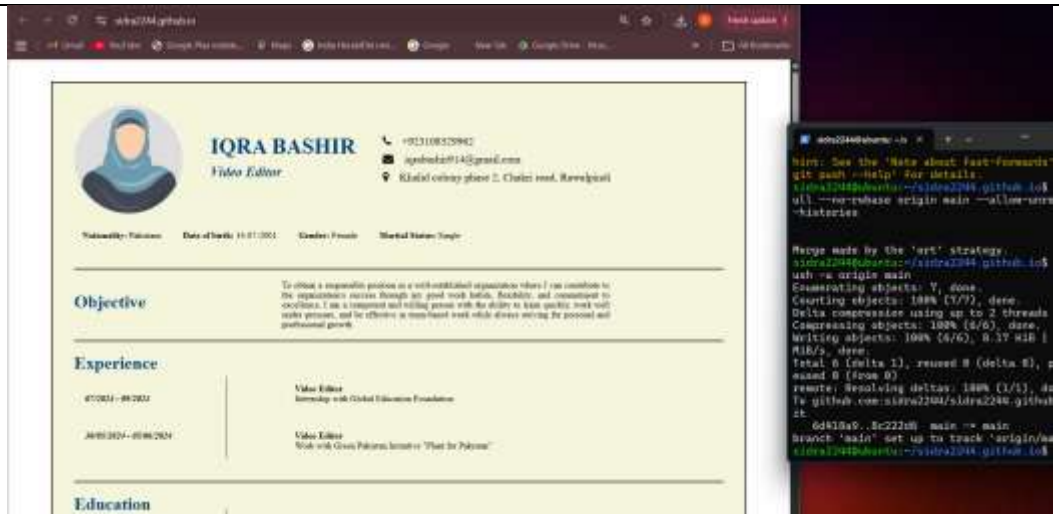
The website files were committed and pushed to the repository.

Screenshot: github_pages_settings.png



GitHub Pages settings showed the publishing configuration and site URL.

Screenshot: live_site.png



The published portfolio website was opened in a browser.

Task Summary

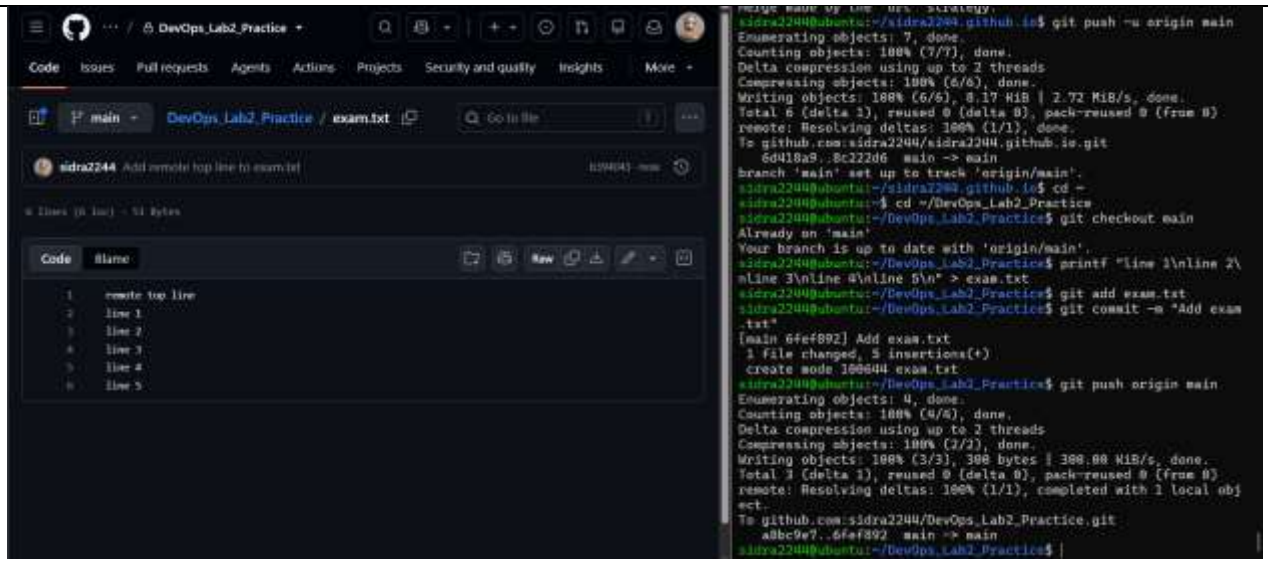
GitHub Pages can host a static portfolio site directly from a public GitHub repository.

Practical Exam / Evaluation Questions

Add the relevant screenshots for the evaluation questions below. Replace each placeholder with your own evidence.

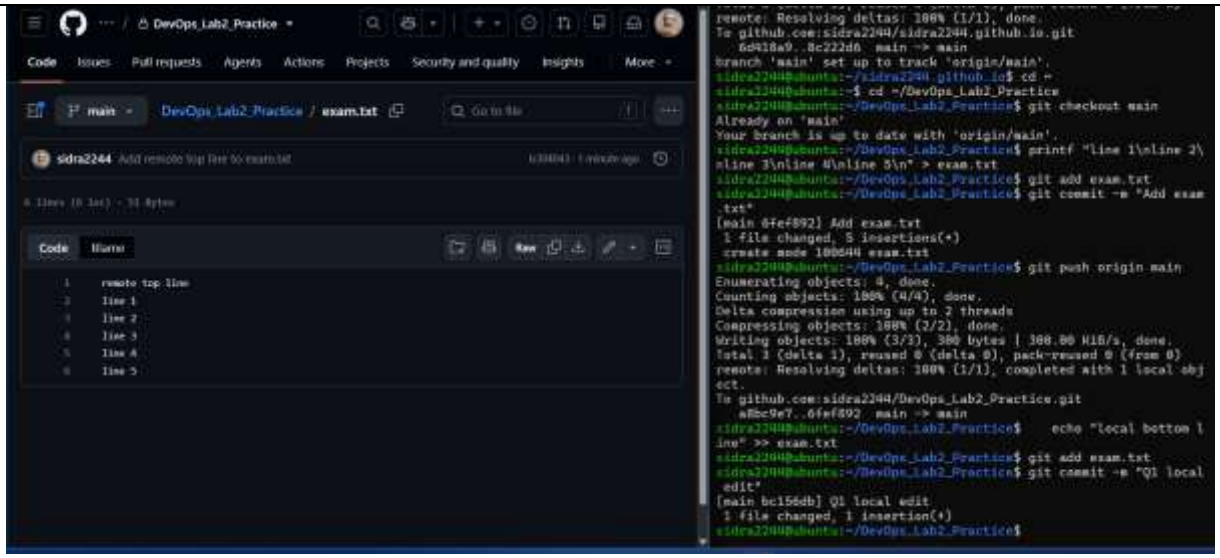
Question 1 – Local vs Remote Conflict Resolution

Screenshot: Q1_remote_edit.png



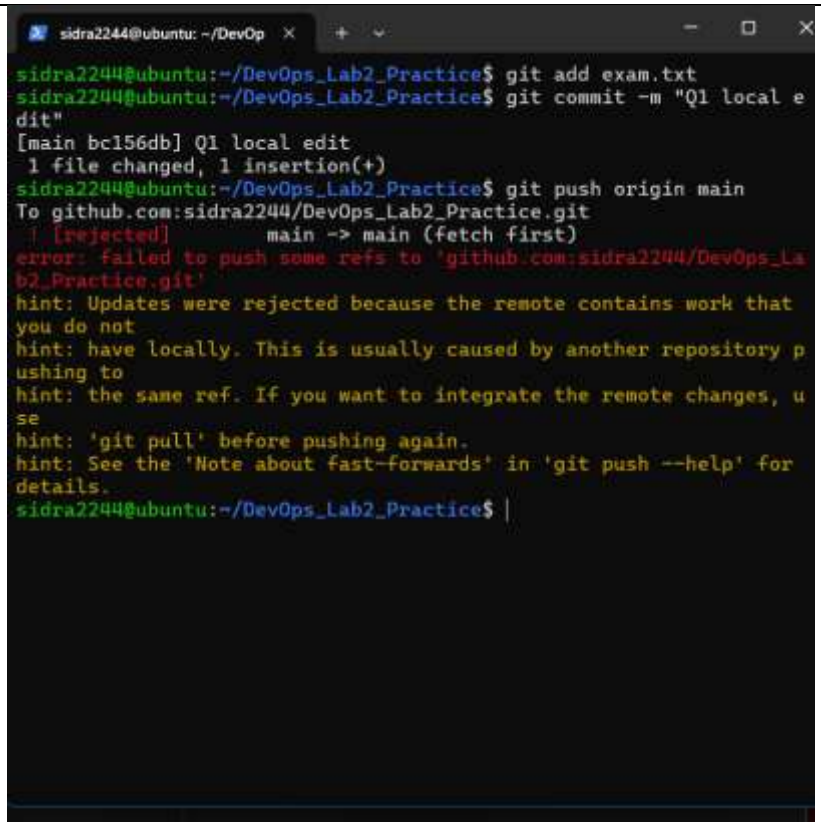
A change was committed directly on GitHub.

Screenshot: Q1_local_edit.png



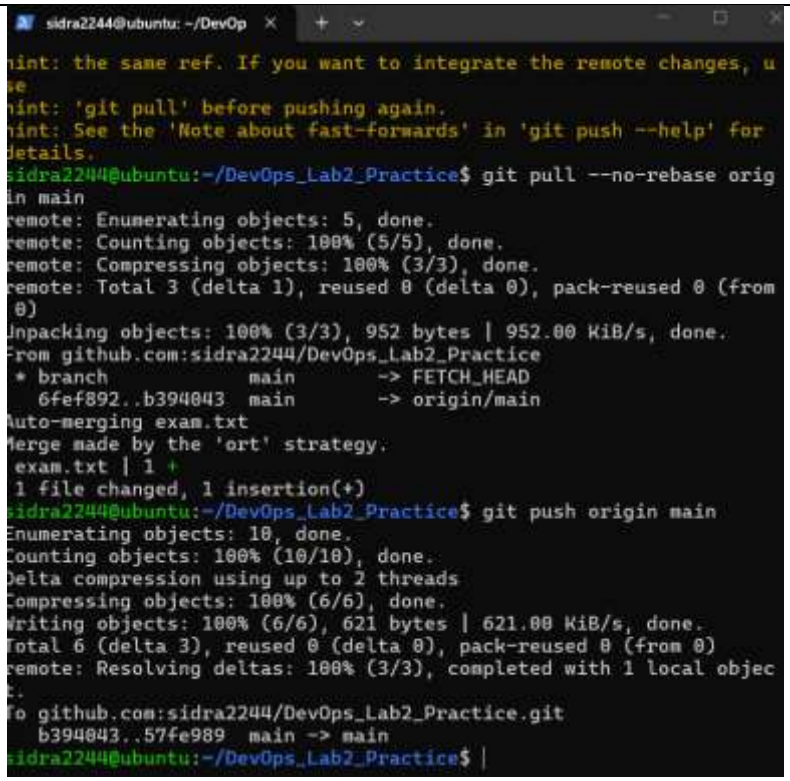
A separate local edit was committed.

Screenshot: Q1_push_error.png



The push error demonstrated that the remote had new commits.

Screenshot: Q1_merge_resolution.png



The conflict was resolved using pull/merge and the result was pushed.

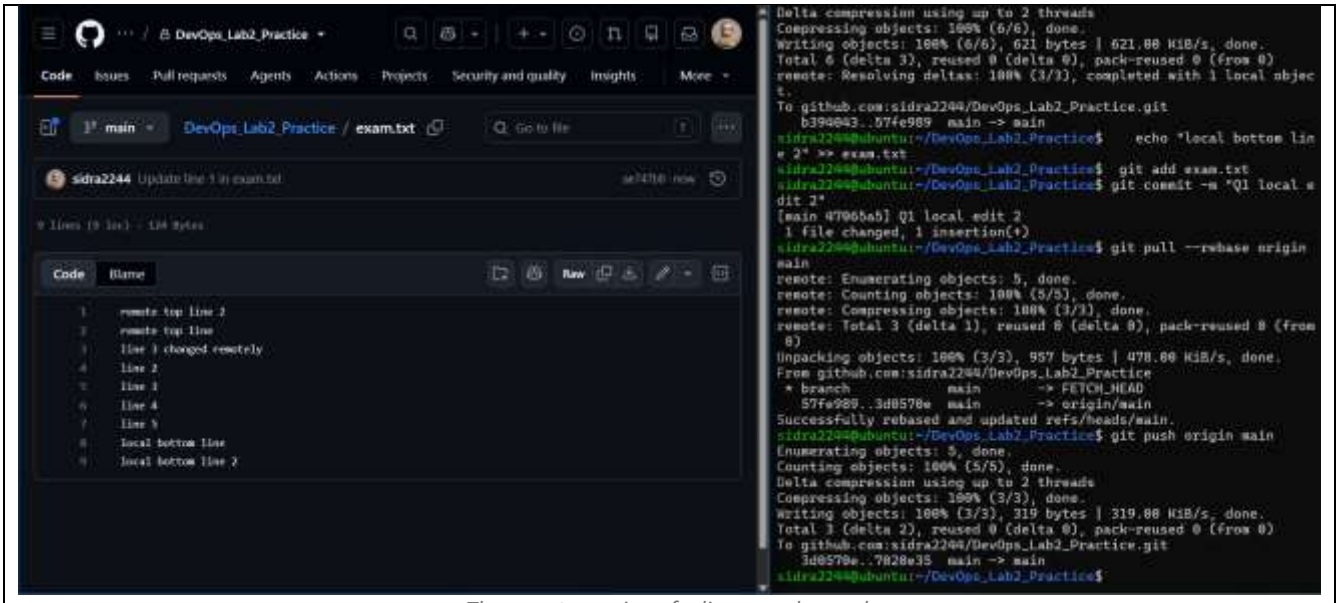
Screenshot: Q1_rebase_resolution.png



The second example was resolved using pull --rebase and pushed.

Question 2 – Manual Merge Conflict Handling

Screenshot: Q2_remote_conflict_edit.png



The remote version of a line was changed.

Screenshot: Q2_local_conflict_edit.png



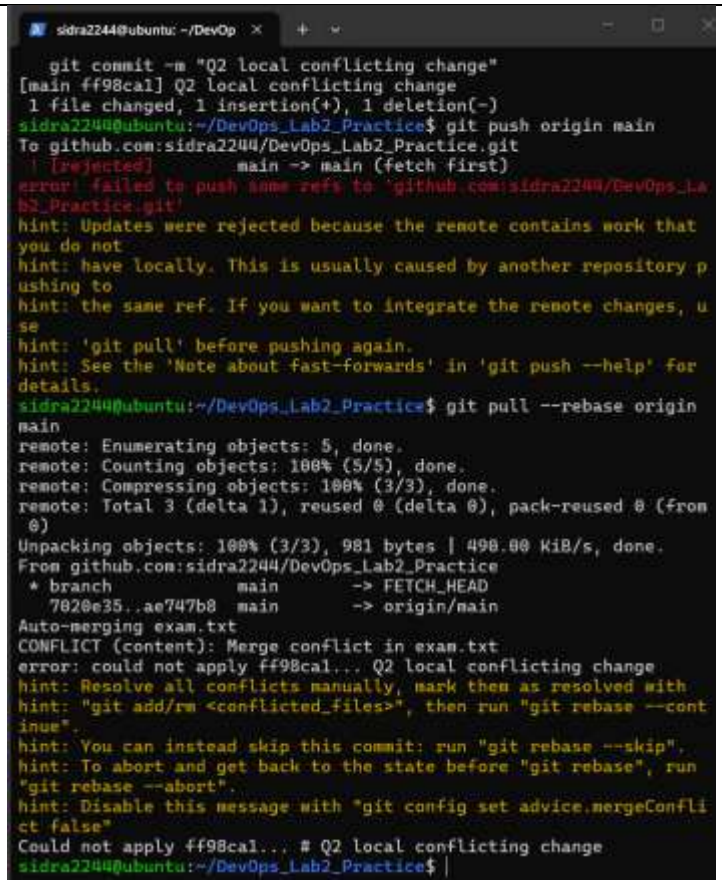
The same line was changed differently locally.

Screenshot: Q2_conflict_push_error.png



The push was rejected.

Screenshot: Q2_rebase_conflict.png



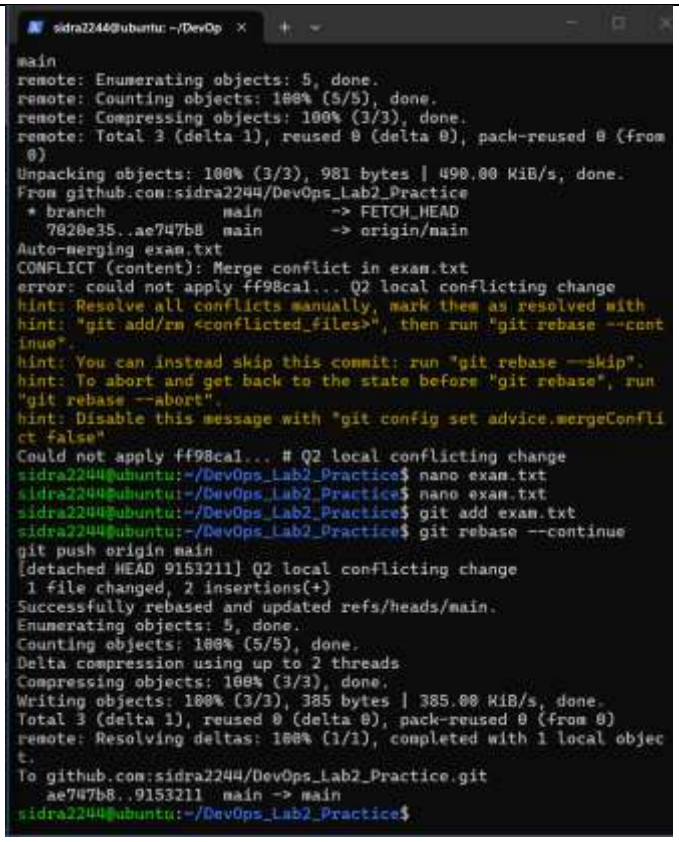
Rebase stopped and reported the conflict.

Screenshot: Q2_resolved_file.png



The conflict was resolved manually in the file.

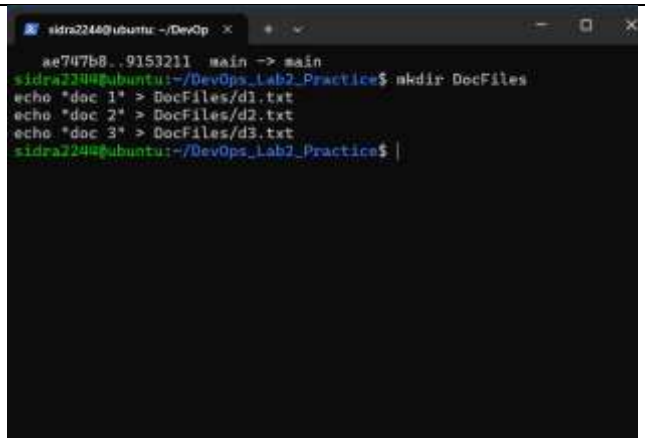
Screenshot: Q2_resolution_complete.png



The resolved change was committed/rebased and pushed.

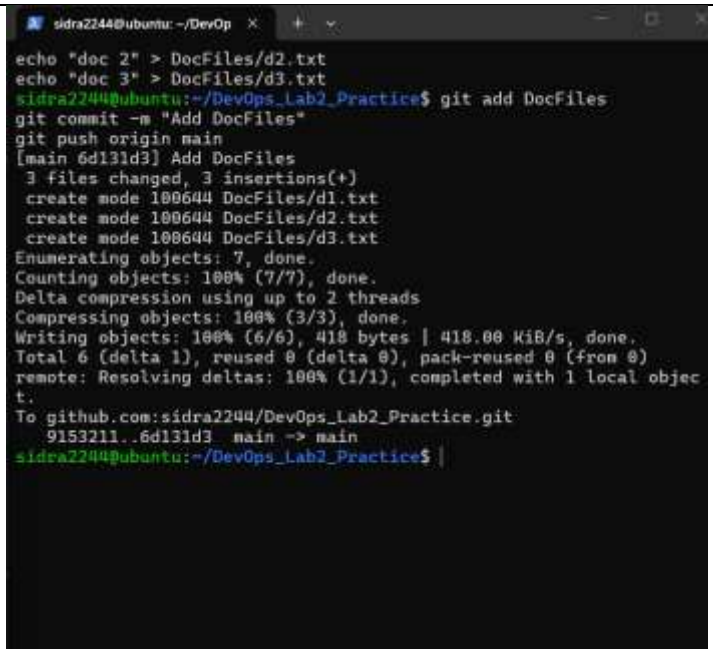
Question 3 – Managing Ignored and Tracked Files

Screenshot: Q3_folder_created.png



A folder with several files was created.

Screenshot: Q3_files_pushed.png



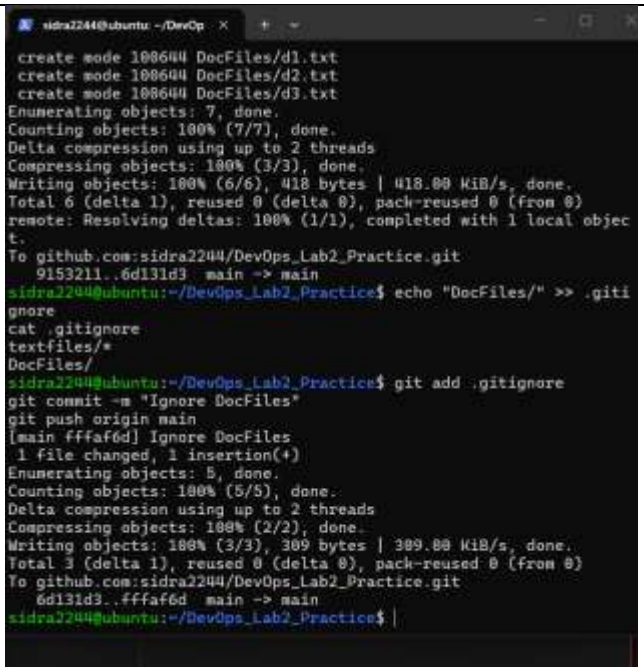
The folder and files were committed and pushed.

Screenshot: Q3_gitignore_added.png



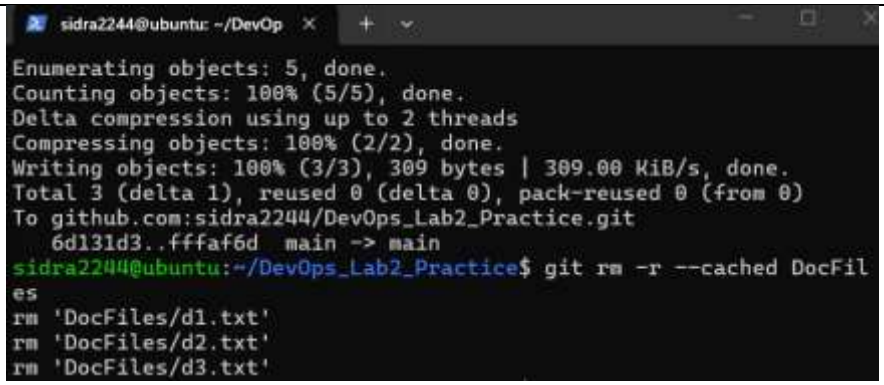
A .gitignore rule was added.

Screenshot: Q3_gitignore_pushed.png



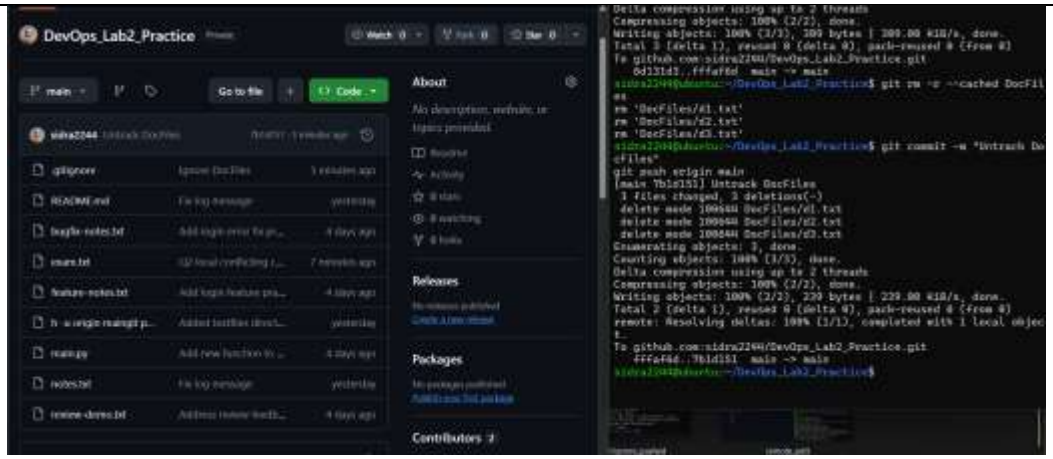
The .gitignore update was pushed.

Screenshot: Q3_folder_untracked.png



The folder was removed from Git tracking with git rm -r --cached.

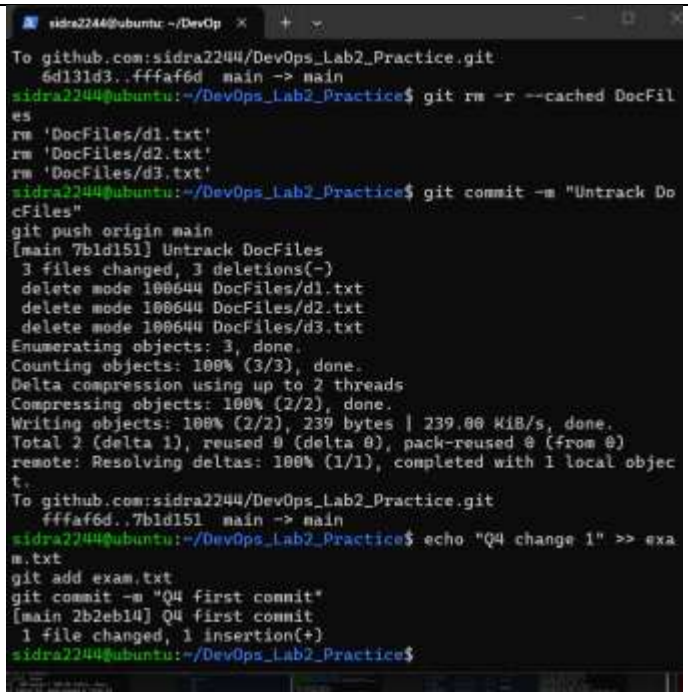
Screenshot: Q3_folder_removed_github.png



The folder was no longer tracked on GitHub.

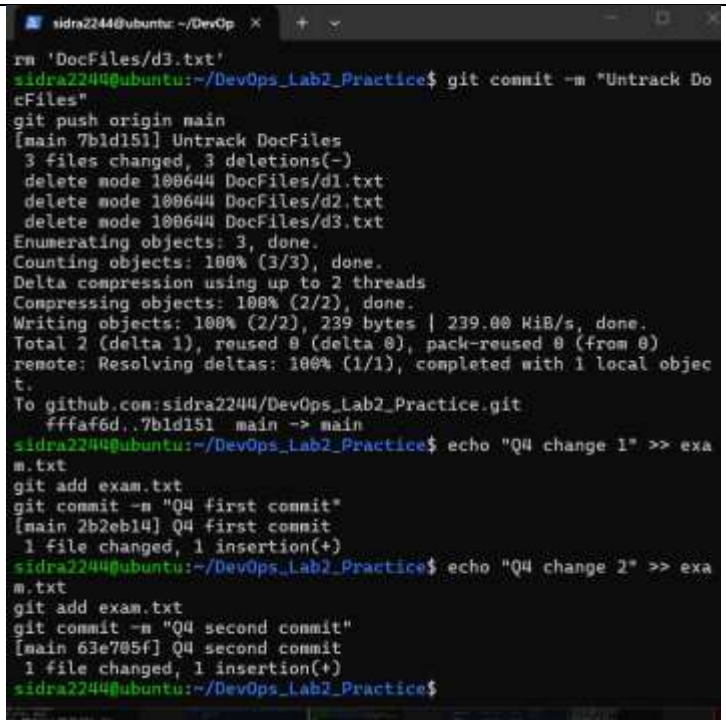
Question 4 – Commit History Manipulation and Recovery

Screenshot: Q4_first_commit.png



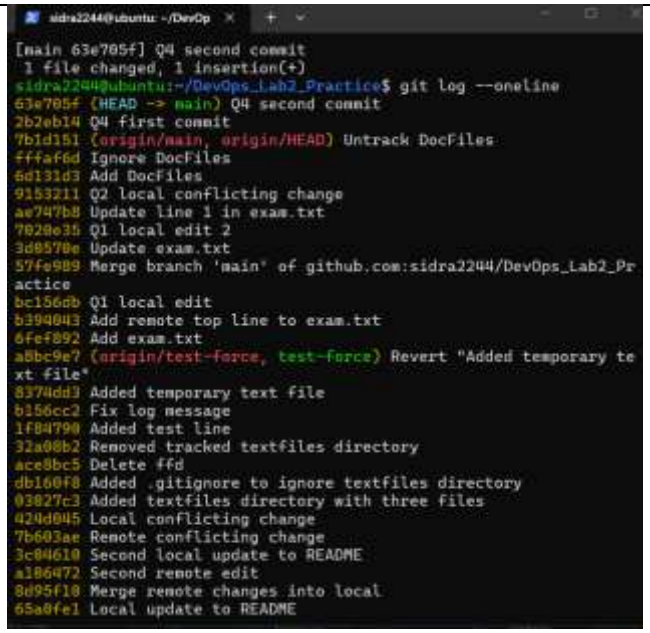
The first change was committed.

Screenshot: Q4_second_commit.png



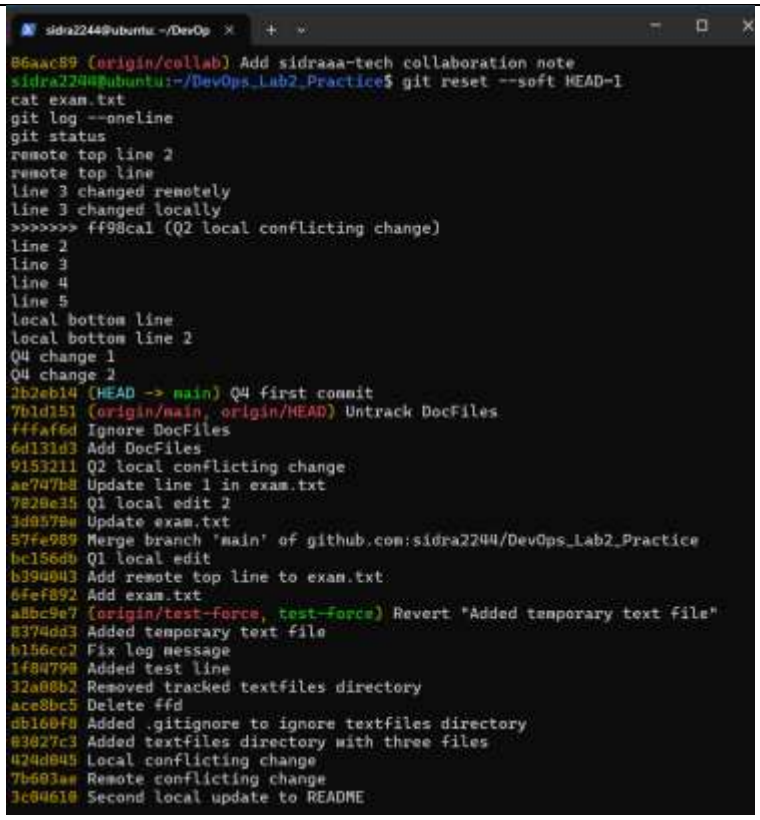
The second change was committed.

Screenshot: Q4_commit_history.png



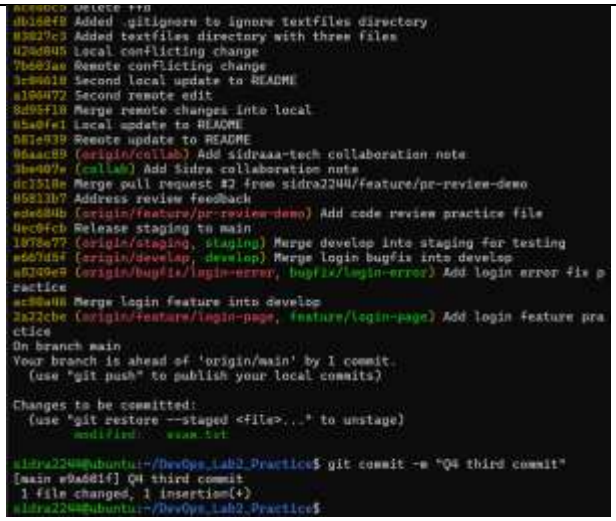
The commit history was displayed.

Screenshot: Q4_soft_reset.png



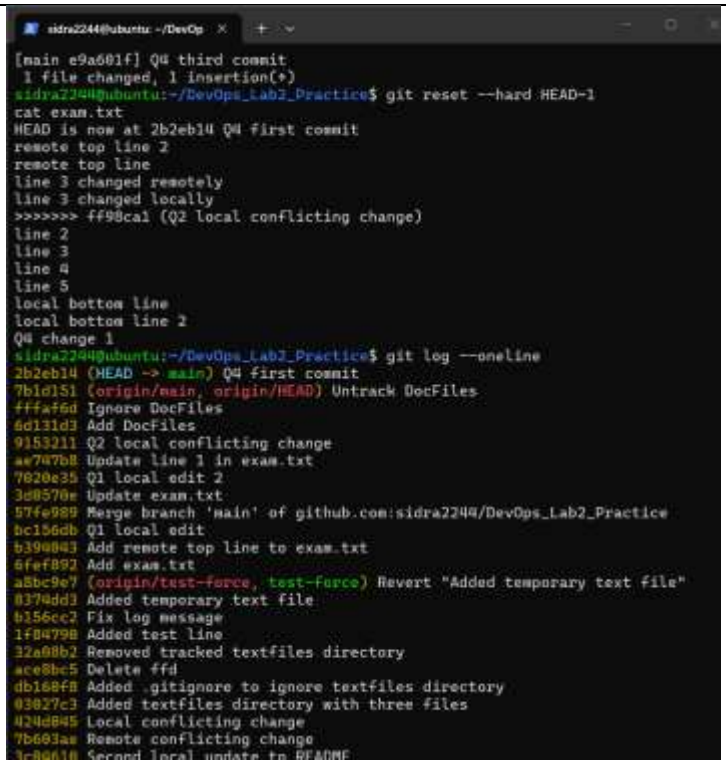
Soft reset was performed and the changes were checked.

Screenshot: Q4_third_commit.png



The change was committed again.

Screenshot: Q4_hard_reset.png



Hard reset was performed and its effect was verified.

GitHub Repository Link

https://github.com/sidra2244/DevOps_Lab2_Practice

https://github.com/sidra2244/DevOps/tree/main/Labs/Lab03